

Mode Sorting with Diffractive Neural Networks

A RESEARCH REPORT

within the framework of a research stay at the University of Oxford

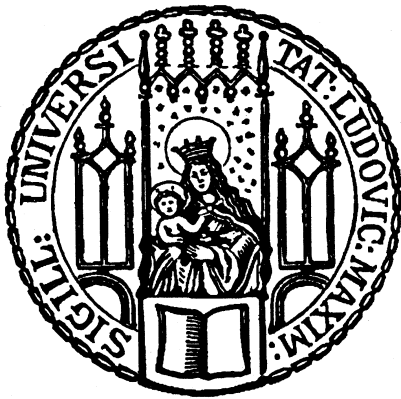
submitted by

TRISTAN ZERWECK

to
the Ludwig-Maximilians-University of Munich

in partial fulfilment of the requirements for the award of the Degree
of
Master of Science
in
Physics

July 2023



Department of Physics
Ludwig-Maximilians-University of Munich



Department of Physics
University of Oxford

Modensortierung mit Beugenden Optischen Neuronalen Netzwerken

FORSCHUNGSBERICHT

im Rahmen eines Forschungsaufenthalts an der Universität Oxford

vorgelegt von

TRISTAN ZERWECK

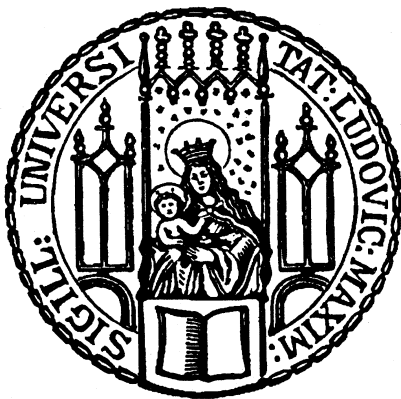
für

Ludwig-Maximilians-Universität München

in teilweiser Erfüllung der Anforderungen für die Verleihung des Grades

Master of Science
in
Physik

Juli 2023



Fakultät für Physik
Ludwig-Maximilians-Universität München



Fakultät für Physik
Universität Oxford

CONTENTS

ABSTRACT	i
LIST OF FIGURES	ii
ABBREVIATIONS	iv
Chapter 1 : Introduction	1
Chapter 2 : Artificial and Optical Neural Networks	2
2.1 Fundamentals	2
2.1.1 Case Study: Classifying handwritten numbers	3
2.2 Mathematics of Neural Networks	4
2.3 Training Techniques	8
2.3.1 MNIST Classification Implementation	10
2.4 Optical Neural Networks	12
2.4.1 Optical Vector Matrix Multiplication	12
2.4.2 Diffractive Optical Neural Networks	13
Chapter 3 : Multi-Plane-Light-Conversion	14
3.1 Applications of Multi-Plane-Light-Conversion	15
3.2 Operating Principle of MPLC Devices	16
3.3 Algorithms for the Generation of Phase Plates	16
3.3.1 Wavefront Matching Method	16
3.3.2 Diffractive Optical Neural Network	18
3.4 Benchmarks of Various Experimental Realizations	20
Chapter 4 : Simulation	21
4.1 Comparison with [1]	21
4.2 Simulation Results for Different Plane Distances	24
4.3 Impact of Beam Waist Size on Results	25
4.4 Comparison with [2]	27
4.5 Free Output Sorting	30
4.6 Simulation Outlook	33

Chapter 5 : Experiment	36
5.1 Setup	36
5.2 Techniques	38
5.2.1 Polarization Calibration	38
5.2.2 SLM Calibration	39
5.2.3 Generation of HG modes	41
5.2.4 Python Interface	41
Chapter 6 : Conclusion	42
ACKNOWLEDGEMENT	43
APPENDIX	44
REFERENCES	

ABSTRACT

Mode sorting is an important tool for novel super-resolution imaging techniques [3, 4], quantum cryptography schemes [5, 6] and telecommunications development [7, 8]. In this report, diffractive neural networks are used to design a mode-sorting device based on multi-plane light conversion. The partial experiment is discussed as well as a simulation of up to 210 modes and multiple diffractive layers and wavelengths. An overall improvement in simulation of the mode sorter's energy efficiency is shown compared to previous research, but a trade-off with crosstalk has to be found. Ideas for improvement of simulation and experiment are given at the end.

LIST OF FIGURES

2.1	Simple structure of three fully connected layers of neurons.	3
2.2	Exemplary images from MNIST dataset.	4
2.3	Different Activation Functions.	5
2.4	Improved version of ReLU	9
2.5	Visualization of Neuron Dropout.	10
2.6	The filtering of a kernel.	11
2.7	Convolutional Neural Network Architecture	12
2.8	Optical Vector Matrix Multiplication. Taken from [9].	13
3.1	Operating Principle of MPLC device for multiplexing HG modes. . .	14
3.2	Visualizing the Wavefront Matching Algorithm.	17
3.3	Overview of Space-Division Multiplexing Techniques.	20
4.1	Simulation results for crosstalk.	22
4.2	Experimental crosstalk results from [1].	22
4.3	Simulation results for loss [dB].	23
4.4	Experimental loss results from [1].	24
4.5	Crosstalk for different plane distances.	24
4.6	Insertion loss for different plane distances.	25
4.7	Crosstalk values for different beam waists.	26
4.8	Efficiency Values from Simulation	27
4.9	Efficiency Values from [2].	28
4.10	Crosstalk Values from Simulation	29
4.11	Crosstalk Values from [2].	29
4.12	Output Mask from Previous Simulations & Reference Papers. . . .	30
4.13	Exemplary Output Mask of Free-Sorting Deep Diffractive Neural Network (D ² NN) with 5 Layers.	30
4.14	Efficiency Values for Soft-Box Simulation	31
4.15	Efficiency Values from [2].	31
4.16	Crosstalk Values from soft-box Simulation	32
4.17	Multi-Plane-Light-Conversion (MPLC) of light between mirror and SLM.	33

4.18	Stretched phase mask.	34
5.1	Simplified Experimental Setup.	36
5.2	Experimental Setup.	38

ABBREVIATIONS

ANN	Artificial Neural Network.
CNN	Convolutional Neural Network.
D ² NN	Deep Diffractive Neural Network.
HG	Hermite-Gaussian.
HWP	Half Wave Plate.
MPLC	Multi-Plane-Light-Conversion.
PBS	Polarizing Beam Splitter.
SLM	Spatial-Light-Modulator.
VMM	Vector Matrix Multiplication.
WFM	Wavefront Matching Method.

CHAPTER 1

INTRODUCTION

Artificial Intelligence is changing the world in more ways than we can think of. Many new tools and applications are emerging that are having a profound impact on the way we live, such as voice and facial recognition tools for leisure and government services or new language models that are slowly entering areas of creative thinking that machines were never thought capable of.

The ever-increasing data and power consumption required to support these technologies is a structural and environmental problem that needs to be addressed urgently. Optical computing has the potential to solve both problems by providing a low-power, ultra-fast computing platform. Optical Vector Matrix Multiplication (VMM)s have already been successfully demonstrated [9], as has the training of optical neural networks [10] via optical VMMs.

In this work, Deep Diffractive Neural Network (D^2NN) are of particular interest because these optical devices are potentially cheaper and more scalable, since spatial-light-modulators, used in many experimental schemes, can be replaced by 3D printed plates. D^2NN s have been shown to successfully sort handwritten digits [11] and can be applied to other tasks outside of computer vision, such as mode sorting. The classification of spatial light modes can be used for numerous imaging techniques [3, 4], quantum cryptography schemes [5, 6] and telecommunication developments [7, 8] (cf Sec. 3.1).

This report first discusses the theory of neural networks and mode sorting via Multi-Plane-Light-Conversion (MPLC), and in the second half presents results from simulations of a D^2NN and first steps of the experimental implementation of such a diffractive neural network.

CHAPTER 2

ARTIFICIAL AND OPTICAL NEURAL NETWORKS

The following pages will explain the basics of both artificial and optical neural networks and give a rough overview of the underlying ideas and mathematical principles.

First, the concept of an Artificial Neural Network (ANN), its architecture and individual components will be outlined. An intuitive explanation is given to make it simpler to grasp how neural networks learn and are able to perform specific tasks. Secondly, the underlying linear algebra and calculus of neural networks is explained to fully understand the training process of ANNs, including forward and backpropagation as well as gradient descent.

The principles of the first part can then be applied to optical neural networks and diffractive neural networks in the following section, whose topics are essential for and merge into the next chapter on MPLC.

2.1 Fundamentals

In simple terms, a neural network can be thought of as a function that takes some input and produces an output. For example, in image recognition, the input values could be the pixel values of an image and the output a specific class defined by the task; if the task of the neural network is to identify objects, the output would consist of several possible classifiable objects (e.g. 3 classes for 'cat', 'mouse', 'dog'). The structure of neural networks is biomorphic, i.e. inspired by biological systems; here the human brain. Fig. 2.1 shows such a simplified architecture. Note that each neuron in a layer is connected to all neurons in the two adjacent layers (hence “fully connected layers”). After the initial activation of some neurons in the first layer (input values), they fire a signal to the connected neurons in the

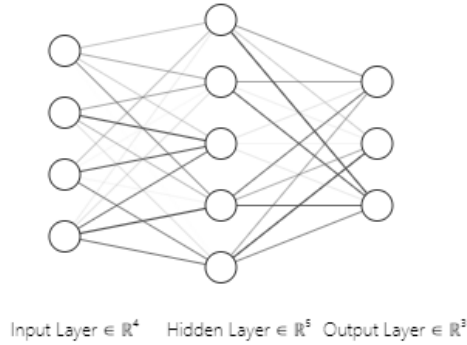


Figure 2.1: Simple structure of three fully connected layers of neurons.

second layer, which in turn pass the signal on to the next layer, and so on until the signal propagates to the end. Each neuron is influenced differently depending on the weights that characterise each connection (the weights are visualised as the opacity of the lines in the diagram). The output neurons are combinations and modifications of the original input signals and represent the solution calculated by the neural network for a given task.

When training the neural network, the weights of the neuron connections are updated at each training epoch until training is complete when the weights converge. The training itself involves the forward propagation of data through the network (also known as inference), then comparing the output predictions of the neural network with specific labels/correct output values via a loss function, and then updating the weights via gradient descent to minimise the loss function. Once all the weights are fully trained, the neural network can be used for inference tasks ranging from image processing to speech recognition to chess and much more [12].

2.1.1 Case Study: Classifying handwritten numbers

It may be unclear at first how digital layers of neurons can perform tasks such as recognising objects, but the following example is intended to explain intuitively how neural networks learn and operate to perform all kinds of tasks.

A canonical task for neural networks in image processing is to classify handwritten single digits from the MNIST dataset. It is often used to compare the performance of different neural networks because the dataset is openly available and machine learning models are highly dependent on the training datasets. Although each handwritten number is unique, neural networks are very effective at learning patterns to identify different numbers in comparison to other machine learning systems such as linear classifiers. These patterns can include objects such as lines,

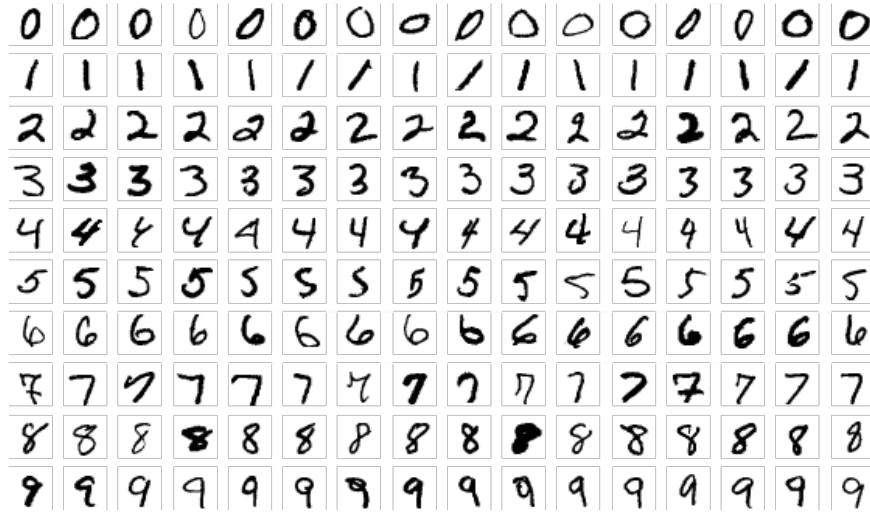


Figure 2.2: Exemplary images from MNIST dataset.
Taken from [13].

circles and semicircles, which together make up the symbols of Arabic numbers. The neural network learns to identify them in the training process by evaluating a cost function, which compares the predicted output number of the neural network with the actual solution (also called the "label"), and then updates the weights of the neurons so that the inference outputs match the labels with greater precision.

However, the patterns that the neural network finds most useful for distinguishing between numbers are often not the lines and circles that are intuitive to humans when distinguishing numbers. Instead, neural networks can identify patterns and characteristics of images at a very abstract level that is not easily understood by humans, as we decompose images into intensities and colours. Neural networks can therefore find very strange-looking interdependencies between different pixels. The performance of ANNs has in many cases been shown to already be beyond human capacity [12].

2.2 Mathematics of Neural Networks

This section illustrates the mathematical description of the data transformations performed inside a neural network to get from the input values to the predicted class, closely following the notation of [12].

First, we define the input data, or the initial layer of neurons, as the vector x . In the case of black and white handwritten digits of 28x28 pixels (MNIST), the first layer consists of 784 greyscale values for each pixel of the image.

The input signal then propagates to the next layer, where each new neuron now

consists of a weighted sum of the original neurons. This can easily be expressed by a vector matrix multiplication $w^l x$ of the weight matrix w^l in the l -th layer and the input values x .

However, further manipulations of the neurons are required to increase the expressiveness of the system. One is to add a bias term $w^l x + b^l$ to each neuron. And the other is to apply a so-called non-linear “activation function” $a^l = f(z^l) = f(w^l x + b^l)$, which is most commonly a sigmoid function, a hyperbolic tangent or a rectified linear unit function, to rescale the data to values between 0 and 1 or -1 and 1 in the case of the sigmoid and tangent functions, or to discard negative values in the case of ReLU. This non-linear activation function is crucial for the learning process of neural networks and is visualised in Fig. 2.3.

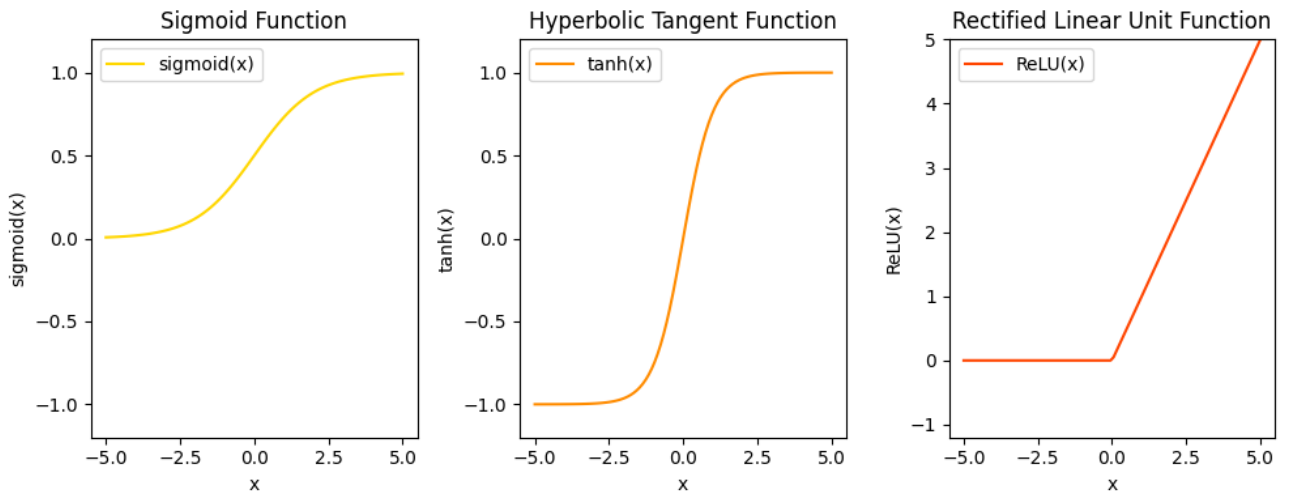


Figure 2.3: Different Activation Functions.

The activation function chosen for a neural network depends on the task. In image recognition, ReLU is most frequently used because it outperforms other activation functions by speeding up the training process by a factor of 33-55 [12].

To recall from the previous paragraph, the final neuron values for each layer are mathematically described by

$$a^l = f(z^l) = f(w^l a^{l-1} + b^l), \quad (2.1)$$

where f is a general activation function. The values of the neurons are then calculated layer by layer, or in other words: the signal propagates through the whole system until it reaches the last layer. This process is called **forward propagation**.

Then the error of the output neurons must be calculated. In the case of handwritten digits, there are 10 output neurons representing each digit from 0 to 9. The neuron that lights up the most, i.e. that has the relatively highest value (between 0 and 1 for the sigmoid function or -1 and 1 for the hyperbolic tangent or 0 to ∞ for ReLU), is the final classification result of the neural network. This is then compared to the actual label of the image, i.e. the number identified by a human on the image.

One possible way to mathematically compare the neural network's output value to the label is the quadratic loss function (mean squared error) which is given by

$$L_Q = \frac{1}{2}(y - a^L)^2, \quad (2.2)$$

where L is the last layer, $l \in [0, L]$, the index Q stands for “quadratic” (loss) and the vector y is the label for the MNIST image; for example, the label for a handwritten 3 would be $y = (0, 0, 0, 1, 0, 0, 0, 0, 0, 0)^T$. The cross-entropy function could, however, also be used as the loss function

$$L_{CE} = -[y \ln(a^L) + (1 - y) \ln(1 - a^L)]. \quad (2.3)$$

Although it seems more complicated at first sight, the usefulness will become clear towards the end of this section.

Having calculated the loss and quantified the performance of these particular weight and bias matrices (which can be randomly initialised at the start), the next step is to update the weights of all the neuron layers in such a way as to reduce the overall loss of the neural network. Therefore, the error δ_j^l of each neuron j in layer l must be found by taking partial derivatives of the loss functions with respect to the specific neuron $\delta_j^l = \frac{\partial L}{\partial z_j^l}$.

Starting from the last layer, the errors for each neuron are $\delta_j^L = \frac{\partial L}{\partial a_j^L} f'(z_j^L)$, using the chain rule. The component-wise expression can be rewritten in vector-matrix form as

$$\delta^L = \nabla_a L \odot f'(z^L), \quad (2.4)$$

where the gradient of the quadratic loss function would be $\nabla_a L = (a^L - y)$. Knowing the error in the last layer, we can compute the errors in the previous layers, i.e. backpropagation of the error - using the following expression:

$$\delta^l = ((w^{l+1})^T \delta^{l+1}) \odot f'(z^l). \quad (2.5)$$

The transposed weight matrix can be intuitively understood as the reverse propagation of the error through a layer, and the Hadamard product in a sense undoes the activation function to give the full error in the l -th layer. Thus, layer by layer, the errors are calculated, which are later used to update the weights and biases of the neurons in a specific way to reduce the loss function.

The individual updates of the weight matrix and bias matrix are computed by gradient descent, where a step of size η is taken in the negative direction of the gradient (direction of maximum ascent) in order to optimally reduce the cost function. The weight matrix in layer l is updated by $w^l - \eta \frac{\partial L}{\partial w^l}$, equivalently the bias matrix is updated by $b^l - \eta \frac{\partial L}{\partial b^l}$. The partial derivatives are simply computed from the previous equations for a^l and δ_j^l and give

$$w^l \rightarrow w^l - \eta \frac{\partial L}{\partial w^l} = w^l - \eta \delta^l (a^{l-1})^T = w^l - \frac{\eta}{m} \sum_x \delta^{x,l} (a^{x,l-1})^T, \quad (2.6)$$

$$b^l \rightarrow b^l - \eta \frac{\partial L}{\partial b^l} = b^l - \eta \delta^l = b^l - \frac{\eta}{m} \sum_x \delta^{x,l}. \quad (2.7)$$

In the final equation, the update of the weight matrix is averaged over m number of input data x , also called a mini-batch. These are used in training to reduce computation, especially when optimising larger models. Note that the previous formulas only apply to the quadratic loss function, which is hidden in the error δ^l .

However, it is sometimes advantageous to use other loss functions, such as the cross-entropy function with $\nabla_a L = \frac{(a^L - y)}{a^L(1 - a^L)}$. The beauty of this loss function comes from calculating $\frac{\partial L}{\partial w^l}$, which simplifies to $x(a^L - y)$. So the weight updates are proportional to the difference between the labels y and the actual neuron values a^L . In other words, the greater the discrepancy (i.e. the worse the network's prediction), the more the weights are updated or the faster the neurons learn [12].

In conclusion, here is a brief summary of the mathematical principles of neural network training.

First, the input data x is fed through the network, where weights, biases and activation functions modify the values according to $a^l = f(w^l a^{l-1} + b^l)$ layer by layer (with different weights and biases in each layer).

Second, the loss function is computed to compare the prediction of the neural network with the label. The loss function depends on the training problem and can be, for example, the quadratic cost function (squared error) or the cross-entropy function (sometimes better understood as a measure of the “surprise” of the error).

Thirdly, the error for the last layer is calculated using Equ. 2.4, and backpropagated via Equ. 2.5 and used to compute the weight matrix updates in each layer via gradient descent, $w^l - \eta \delta^l (a^{l-1})^T$, and the bias matrix updates, $b^l - \eta \delta^l$, respectively. The updates are applied and the whole process is repeated from step 1.

2.3 Training Techniques

In practice, artificial neural networks are a vast experimental playground. Empirical methods must be exhausted to find the optimal architectures and parameters for different computational tasks. In image recognition, for example, convolutional neural networks have been shown to identify patterns and features of objects with much higher efficiency than fully connected layers. Not only the chosen architecture, but also the activation function, the loss function and many other details can change the performance of a system enormously. Therefore, the following pages provide a brief overview of some useful techniques for increasing the efficiency of ANNs in image recognition tasks.

First, as mentioned above, the right loss function is crucial. Although the quadratic cost function seems reasonable at first, the cross-entropy loss function is very advantageous because the neurons learn faster when producing larger errors. Some other additions can also be made to the loss function, such as regularising the cross-entropy by an extra term at the end (called L2 regularisation):

$$L_{CE,L2} = -[y \ln(a^L) + (1 - y) \ln(1 - a^L)] + \frac{\lambda}{2} w^2. \quad (2.8)$$

This is called the regularised cross-entropy function. The extra term at the end is minimal for small weights and maximal for large weights. Therefore, these two terms must be balanced to minimise the loss, depending on the value of λ . For larger λ , the right term becomes more important and the weights become smaller. Conversely, for smaller λ the loss function is minimised more and the weights are larger. Overall, this increases ANN performance and reduces problems with overfitting (see [12]), which is a term for the wasteful process of training a neural network after it has converged to the optimal parameters.

Another very similar regularisation technique is called L1 regularisation, which includes an additional term that also penalises large weights:

$$L_{CE,L1} = -[y \ln(a^L) + (1 - y) \ln(1 - a^L)] + \lambda|w|. \quad (2.9)$$

Although the similarities are striking, the effect is quite different: many weights are driven to values close to zero, while only a few essential connections survive.

Second, activation functions can also play an important role in the training process. A common problem in training is the vanishing gradient problem. While changing the activation function from sigmoid to ReLU might help, in some cases the “leaky ReLU” is even better, because it does not set some weights directly to zero, but to a small negative number, which causes them to be updated and not to get stuck at zero.

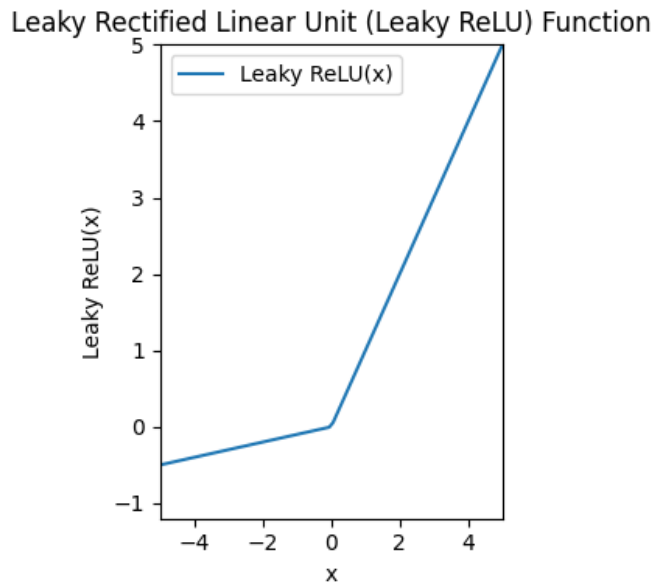


Figure 2.4: Improved version of ReLU

Thirdly, another interesting method of improving the performance of a neural network is dropout. The next figure (Fig. 2.5) visualises the simple principle of dropout, where some neuron connections are cut or, mathematically, some weights are set to zero in a training period. This proves effective in some training

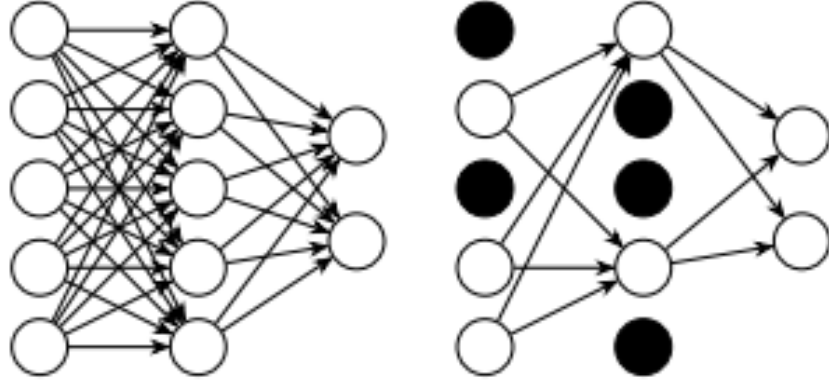


Figure 2.5: Visualization of Neuron Dropout.
Taken from [14].

schemes, where dropout can be thought of as a technique for creating different neural networks. Therefore, backpropagation and error computation is like taking the average of multiple neural networks, thus minimising overfitting.

Other methods such as choosing different optimisers, gradient descent (e.g. stochastic gradient descent (SGD) or adaptive moments (ADAM)), or increasing the training data sets are also possible. Artificially augmenting the training data is often easier than collecting more data in other ways. Augmentation methods in image recognition include cropping images, slightly rotating them, changing colours, etc., depending on the model and the task. For obvious reasons, these larger training sets make the model more robust and versatile, increasing classification accuracy.

2.3.1 MNIST Classification Implementation

This section shows how a simple neural network can be improved by adding the techniques presented earlier to classify MNIST handwritten digits.

A single hidden layer neural network with layer dimensions (784, 100, 10) was set up and trained for 10 epochs. With the correct learning rate (0.001), both ADAM and SGD gave an accuracy of 95%. Adding another 50000 randomly rotated images (rotations up to 5 degrees) to the training data (50000 images) increases the accuracy to 98.21%. Small changes were achieved by improving the optimiser's weight decay parameter, setting it to $1e-9$ (= 98.28%). Furthermore, a small neuron dropout of 5% increased the accuracy to 98.34%. Training this

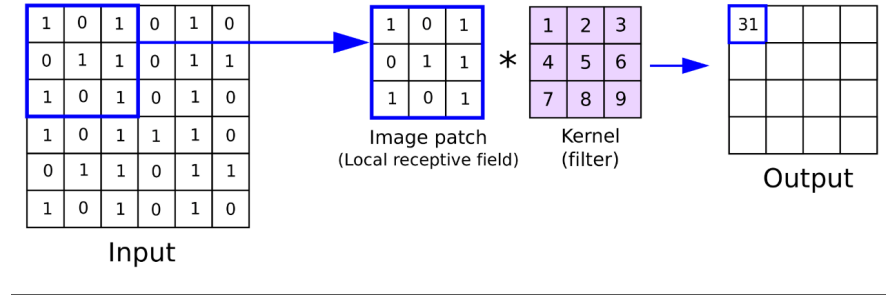


Figure 2.6: The filtering of a kernel.
Taken from [15]

neural network for 500 epochs brought small improvements, i.e. an accuracy of 98.40%.

Even better results were obtained with a different architecture, namely a Convolutional Neural Network (CNN), as shown in Fig. 2.7. This architecture is more complex and can therefore filter out more features on an abstract level. A rough explanation will be given in the next paragraph to understand the concept of CNNs.

First, the input data is passed to the first convolutional layer. Distinctly, this is not done by weighting the neurons, adding a bias and applying the activation function, but by “scanning” the image with a so-called kernel. This kernel looks at a square of pixels (in this case 5x5) of the original input image and then weights and sums these pixel values to compute a neuron value for another pixel/neuron on the feature map, see Fig. 2.6. The kernel sequentially analyses all 5x5 pixel squares in the image and creates a corresponding value on the feature map. Thus, it is the kernel that decides how and what information is passed on to a particular feature map (i.e. different kernels produce different feature maps and analyse different features or patterns of the image).

The convolutional layers themselves consist of different feature maps created from different kernels. In the next layer, called the max-pool layer, another 2 by 2 square pixels are scanned from the previous layer of (in this case) 6 transformed images. This time the “scanning” is not about weighting and summing, but just taking the maximum value of these 4 pixels (hence the “Max-Pool” layer). This reduces the computational load and summarises the information to the required (maximum) values. This is followed by another convolutional layer and a max-pool layer, which further dissect the features of the image, and finally 3 fully connected layers, which reduce the information to the 10 output classes needed.

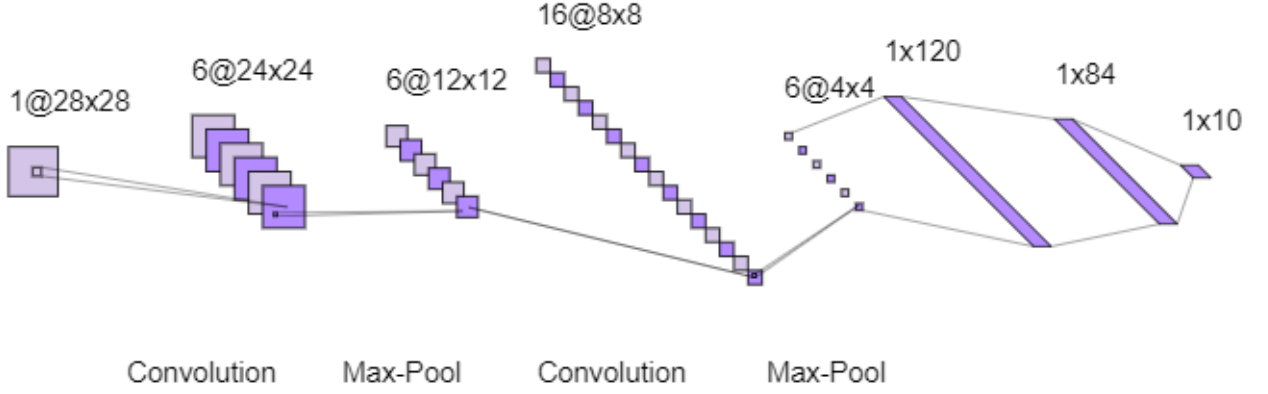


Figure 2.7: Convolutional Neural Network Architecture consisting of 2 convolutional layers, 2 pooling layers and 3 fully connected layers.

Much better results were obtained with this architecture. First, an accuracy of 98.46% was achieved with 6 training epochs, a learning rate of 0.0005 and augmenting the training data with rotated images (5 degrees). After adjusting the number of epochs (10 or 7) and the learning rate (0.01 or 0.015 respectively), the best accuracy of **99.01%** was obtained using Stochastic Gradient Descent (SGD) as the optimiser (and not ADAM).

2.4 Optical Neural Networks

In recent years, methods have been developed to perform classical computation with optics using light fields by modifying their wavefronts and amplitudes, thus encoding and transforming information with light instead of using standard computer chips [11]. Optical computing is a promising candidate for a faster and more energy-efficient data processing method of the future.

2.4.1 Optical Vector Matrix Multiplication

The basic building block of any neural network is vector matrix multiplication, both in forward and backward propagation. It has recently been shown that handwritten digits from a downsampled version of the MNIST image dataset can be classified by an optical inference process using real-valued matrices [10]. Optical vector matrix multiplications were performed to transform the input neurons v into the output neurons u by applying a weight matrix W such as $u = vW$, as described in more detail in earlier sections.

The experimental process is as follows: an optical field hits a Spatial-Light-Modulator (SLM), which is a device that modifies the amplitude and phase of light in small areas (pixels on the SLM) and can thus encode information in the

whole light wavefront. The first SLM encodes different copies of the input vector into the light field, as shown in Fig. 2.8. This light field then propagates to the next SLM, which multiplies the incoming light field with the weights ¹ of W by transmitting a certain fraction of the light. The transformed field then passes through a cylindrical lens, which sums the values horizontally, completing the optical vector matrix multiplication. The output light field is then captured by a camera and is the equivalent of the output neurons of a digital neural network [9].

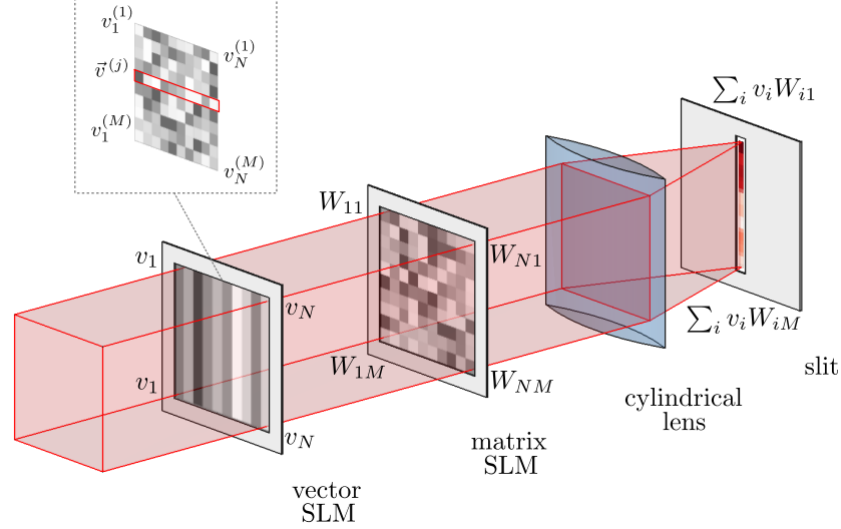


Figure 2.8: Optical Vector Matrix Multiplication. Taken from [9].

2.4.2 Diffractive Optical Neural Networks

Diffractive optical neural networks work in a slightly different way. There are no full vector-matrix multiplications performed, instead the output neurons y are defined by element-wise vector multiplications of the input values x with propagation and transmission vectors P, T , like $y = P \odot T \odot x$ where the Hadamard product is denoted by $\odot$.

There are no copies made of the initial vector x and there is no final cylindrical lens to sum up the element-wise multiplications, but otherwise the setup is the same as in the figure above, where element-wise multiplications are realised by reflections from spatial light modulators. The successful implementation of deep diffractive neural networks for the classification of handwritten digits can be found in [11].

¹The weights are between 0 and 1 if you are modulating only amplitudes with the SLM for real-valued weight matrices. However, the values can also range from -1 to 1 when the SLM modulates phases for complex-valued weight matrices.

CHAPTER 3

MULTI-PLANE-LIGHT-CONVERSION

Modifying the shape of light is a powerful tool that can be applied to many different technologies, such as telecommunication, astronomy and microscopy, see Sec. 3.1. In general, MPLC can be used to alter the spatial profile of a light field. Multiple planes scatter the light, introducing deliberate phase or amplitude shifts to create desired output fields, and are capable of performing arbitrary linear wavefront transformations. The efficiency of the transformation is generally limited by the number of phase plates. In the case of mode-sorting, however, there exist some methods of reducing the number of plates required (discussed in more detail in Section 3.3).

In the following pages a general overview of MPLC devices and its abilities, especially with regard to mode sorting, will be presented, as well as a comparison of various experiments and their achieved benchmarks in order to relate several results to our system.

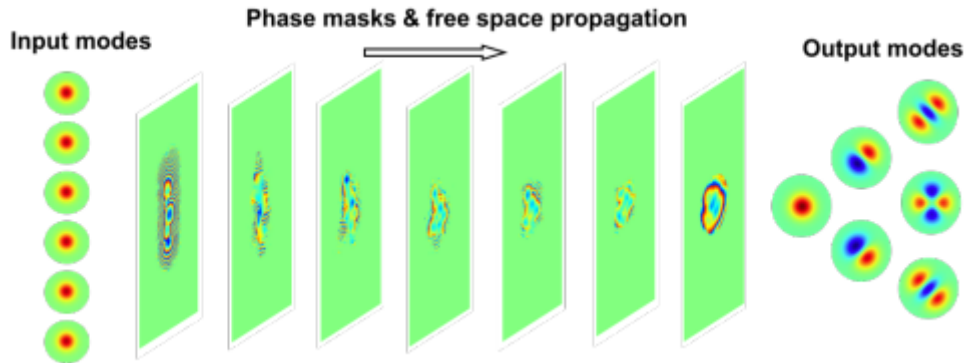


Figure 3.1: Operating Principle of MPLC device for multiplexing HG modes. The input modes propagate through free space and the phase plates. They are modified and transform into the desired output modes. Taken from [16].

3.1 Applications of Multi-Plane-Light-Conversion

There are numerous applications for MPLC devices. A major commercial interest is in telecommunications and the extension of transmission bandwidths by processing information in higher order modes in multi-mode fibres (MMF). Beam shaping for spatial multiplexing/demultiplexing of modes is crucial for this task and has been demonstrated in many experiments [7, 8, 17, 1, 18]. Since multi-plane-light-conversion can sort any orthogonal set of modes, including Laguerre-Gaussian (LG) modes, Hermite-Gaussian (HG), linearly polarised (LP) modes, orbital angular momentum (OAM) modes, this opens up a wide range of applications for spatial filtering of light fields in other areas such as (superresolution) microscopy and non-invasive imaging applications such as astronomy [19].

Furthermore, quantum cryptography schemes based on controlling spatial light modes exist [5, 6, 20, 21]. There are also several quantum optical computing experiments, such as [22, 23], including the realisation of quantum gates (Hadamard and Pauli gates) by sorting spatial modes of photons for quantum computing [24], bringing us a tiny step closer to quantum communication and computing.

Other use cases include imaging through randomly scattering diffractive media and demultiplexing light modes propagating through a multi-mode fibre. This could be applied, for example, to imaging through thin surfaces (biomedicine, atmospheric physics, etc.), broadband telecommunications, and localisation-based or parallelised super-resolution microscopy [25, 26].

More recent applications also include far-field superresolution imaging techniques, which decompose optical fields into Hermite-Gaussian spatial modes, e.g. for astronomical or non-invasive biological imaging purposes [3, 4].

MPLC has also been used to sort wavelengths, which is potentially useful for high-power wavelength beam combining (WBC) or wavelength division multiplexing (CWDM) [27].

Finally, MPLC devices have been able to perform neural network tasks such as optical sorting of handwritten digits [11] and could be useful for other object recognition services in computer vision, which are of particular interest in this report and experiment.

3.2 Operating Principle of MPLC Devices

MPLC devices shape light wave fronts by applying small phase shifts $E(x) = e^{i\Delta\Phi(x)}$ to different positions x of the optical field. Together with diffraction by free space propagation, this gives the desired output field

$E_o(x) = A(x)e^{i\Phi_i(x)+i\Delta\Phi(x)}$, where the indices o and i stand for output and input, respectively. The working principle is the same as for diffractive neural networks, explained in section 2.4.2. More about the generation of the right output field by wavefront modulation on a SLM can be found in section 5.2.3.

The resolution of the generated wavefront is limited by the lithographic fabrication limits of the phase masks or SLM resolution limits. Most experiments use phase masks with a pixel size of a few microns up to 15 μm .

3.3 Algorithms for the Generation of Phase Plates

3.3.1 Wavefront Matching Method

The dominant algorithm for producing phase masks is the Wavefront Matching Method (WFM) [1, 19, 26], which was originally used for waveguide design [24]. The algorithm overlaps the input and output mode electric fields $\Psi_{i/f}(x, y, n)$ at a given pixel (x, y) of a given plane n . The overlap at plane n is given by $o(x, y, n) = \Psi_i(x, y, n)\Psi_f(x, y, n)e^{i\Phi_n(x, y)}$, where $\Phi_n(x, y)$ characterises the phase shift induced by the n -th phase mask. Therefore, the phase correction for the n -th plane, for the specific pixel (x, y) and for all K modes can be calculated as

$$\Delta\Phi_n(x, y) = -\arg\left(\sum_{k=1}^K o(x, y, n)_k\right). \quad (3.1)$$

The modified wavefront then propagates to the next phase plane, where the optical field is again superimposed on the output modes and a certain phase shift is calculated, so that the output modes are iteratively forced onto the propagating wavefront to align it with the desired result, see Fig. 3.2. This is done for all the phase plates in the forward direction first (starting with the first phase plate, then the second, the third, etc.) and then in the backward direction, calculating the overlap at the last phase plate, then the second last plate, and so on until the first is reached again. This process is repeated until the phase patterns on the tiles converge. No gradients are computed in this algorithm, unlike the Gerchberg-Saxton algorithm for generating single phase plates or a diffractive neural network. It is worth noting that the WFM is not able to optimise the pre- and post-MPLC

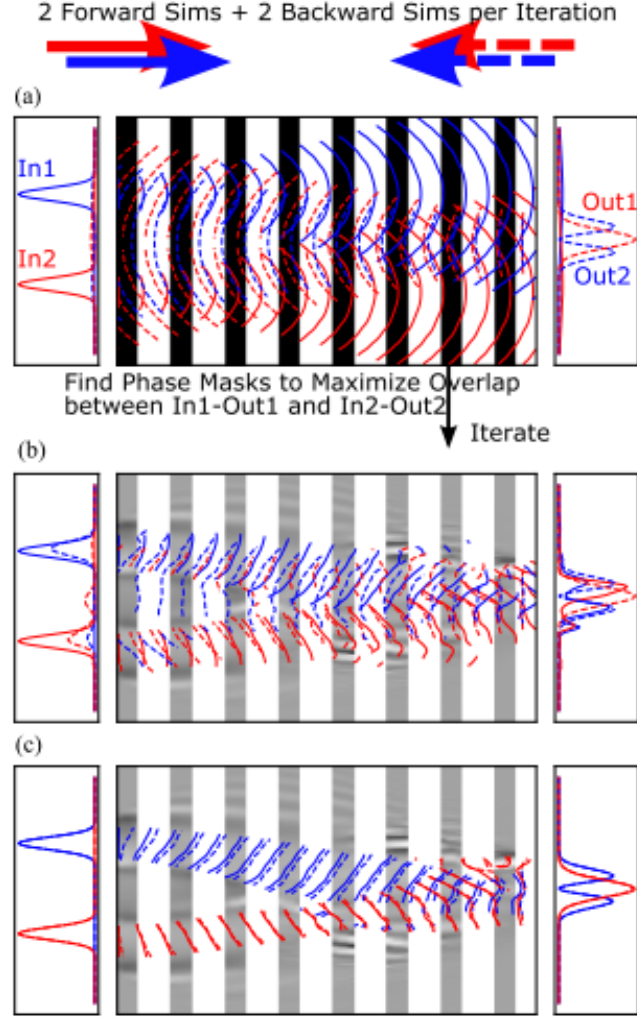


Figure 3.2: Visualizing the Wavefront Matching Algorithm.

Conversion of two Gaussian beams into 0th and 1st order HG-modes. Forward propagating modes are solid lines, backward modes are dashed. **a)** initial phase plates allowing undisturbed propagation. **b)** The light propagation after some iterations of optimizing the phase plates. **c)** Final propagation.

Taken from [28].

distances, the mask smoothness, the number of phase planes and the mode dependent loss, instead certain ML techniques have to be used to optimise these [28] or a completely different algorithm than WFM has to be chosen, such as D²NN. For a brief mathematical overview see [24] and for further details on the mathematics of inter-plane propagation see the angular spectrum method and Ref. [29].

Significant improvements to the WFM have recently been made by taking further effects into account when calculating the phase plates (see [2]). In this paper, the objective function is not just the overlap between input and output modes, but also includes further crosstalk and mode sorting efficiency terms, which significantly improve the performance of the simulated MPLC device for sorting HG modes.

3.3.2 Diffractive Optical Neural Network

A new approach to creating phase plates has been explored recently, which uses neural networks to compute phase plates for multi-plane light conversion: in essence, each layer of neurons translates into a phase plate and the free space propagation together, which transforms the incoming light field to create the desired output field.

The potentially biggest breakthrough in mode sorting with a D²NN is the reduction in the number of phase plates through “free output” sorting. To explain what this means, it is necessary to take a step back and look at previous mode sorting research.

There is no rigid theory explaining the minimum number of phase masks required for any arbitrary optical transformation. Some authors claim that sorting N modes generally requires N number of phase plates [16, 19] or even $2N$ [24]. In 2019, however, a milestone in mode sorting was reached when it was shown that 325 modes could be sorted with only 7 phase masks [1] (and later, in 2021, even 1035 modes were sorted with 14 masks [30]). This rapid increase in mode numbers was attributed to a special arrangement of input and output modes, namely the triangular arrangement of the output Gaussian rays, and was claimed to be one of several configurations that allowed a large number of modes to be sorted with only a few phase masks. Similarly, there is no rigorous theory to explain why this arrangement is advantageous or what other possible configurations exist.

Nevertheless, diffractive neural networks may help to get a step closer to solving this problem. Although D²NN will not theoretically characterise the whole possible configuration space, it may well find other solutions to optimally assemble the input/output modes to further reduce the number of phase masks. This can be done by allowing the neural network to freely choose the output mode configuration by adjusting the loss function.

The old loss function from WFM can be expressed by

$$L = \frac{1}{K} \sum_{k=1}^K (1 - \eta_k), \quad (3.2)$$

where η describes the overlap between input and output modes

$\eta_k = |\Psi_i, k * \Psi_f, k|^2$ and with the indices i and f for the input (initial) and output (final) modes, plus the k index to represent a particular mode (e.g. HG mode 0,0). This loss function is unfavourable because it imposes constraints on the unitary

transformation as the output modes are predefined.

Instead, the loss function should be given by

$$L = \frac{1}{K} \sum_{k=1}^K \sum_{j>k}^K (1 - \eta_{k,j}), \quad (3.3)$$

where $\eta_{k,j}$ describes the overlap between the k th and j th input mode $\eta_{k,j} = |\Psi_{i,k} * \Psi_{i,j}|^2$. The system thus minimises the overlap of input modes and finds a configuration where the modes are optimally spatially sorted for a given number of phase plates. WFM cannot minimise this loss function, whereas D²NN can.

The gradients of the phase masks Φ_n are computed with respect to the loss function (automatic differentiation) and the phase masks are updated iteratively over $\Phi'_n = \Phi_n - \gamma \nabla_{\Phi_n} L$. When using optimisers such as ADAM for a neural network architecture, an additional momentum scaling factor would be added to the gradient term, which is not explicitly written here for simplicity. As noted above, a big advantage of D²NNs is that you can also optimise any other parameter such as the distances between the phase plates z_n via $z'_n = z_n - \gamma \nabla_{z_n} L$.

Another great advantage of neural networks is that the pixels of the phase plates are updated in parallel (and not sequentially as in WFM). Neural network optimisers, such as ADAM, calculate the gradient of the loss function by automatic differentiation and apply individual changes to all neurons in a layer simultaneously.

Furthermore, many other parameters of the MPLC system, such as the light propagation distances (before, between and after the phase plates), can be chosen as free optimisation parameters and thus easily adjusted, whereas in WFM they are fixed [29].

In addition, batch training allows a significant reduction in computational complexity, as shown in [29]. Finally, as the use and development of neural networks has grown exponentially in the last decade, many Python libraries are readily available for optimising neural network training processes, making this a great platform for computing phase plates.

For example, augmenting training data sets (adding noise, cropping images, etc.) requires only one line of code in pytorch, which is useful for making MPLC devices more robust to experimental errors.

3.4 Benchmarks of Various Experimental Realizations

When using MPLC for mode sorting, an important measure of performance is crosstalk (defined slightly differently in each experiment) and total light loss in the system, as well as efficiency, in terms of how much energy from the input mode ends up at the output mode.

To characterise the performance of our system, the results are compared with some of the experimental values from other groups in the following table.

Comparison of Experimental Results					
Paper, Year	[31], 2017	[24], 2019	[1], 2019	[30], 2021	[26], 2022
Nr. of Modes Sorted	10	5	210/325	1035	403
Nr. of Phase Plates	5	3	7	14	15
Phase Plate Size	512x512	630x630	500x274	512x256	400x400
Pixel Size [μm]	3	8	8	10	12.5
Input Size (Gaussian) [MFD in μm]	n/a	n/a	60	31.2	n/a
Output Size (HG mode) [MFD in μm]	n/a	n/a	400	300	n/a
Average Cross Talk	-26dB	-6dB	-7.2dB	-12.8dB	-6dB
Insertion Loss	4dB	n/a	6dB	2.1dB	n/a

Note that all the experiments above use the WFM to compute the phase masks. The results presented here are among the most effective, using very few phase planes realised either by spatial light modulators (SLMs) or by lithographically etched glass substrates with dielectric coatings.

It is also worth comparing MPLC results with other space division multiplexing methods: the following table from [28] gives an interesting overview of the field.

Device	Best Application	Scalability	Fabrication Maturity	Loss	Bandwidth	Size
Photonic Lantern	low loss and broadband	100s of fibers	labor intensive	< 1 dB	100-500nm	a few cm long
MPLC	complex modes	1035 HG modes	waferscale	> 3 dB	100 nm	100 mm ³ to 10 cm ³
3D Waveguide	spot remapper	100s of cores	mature	1 dB	>100 nm	10 mm×10 mm×1 mm

Figure 3.3: Overview of Space-Division Multiplexing Techniques.
Taken from [28].

CHAPTER 4

SIMULATION

A digital diffractive neural network developed by Matthew Filipovich in the Quantech group was used in the following chapter to analyse certain characteristics of an MPLC system and to compare its performance with the wavefront matching method for Hermite-Gaussian mode sorting. The neural network can solve mode sorting tasks for different outputs. The two relevant cases are sorting modes into output boxes arranged in a triangle, as mentioned in the previous chapter, and also sorting into “soft” output boxes, or “free” output boxes, where the network chooses its own arrangement.

4.1 Comparison with [1]

In this section, a comparison of the sorting of 210 modes into boxes arranged in a triangle was made to compare Filipovich’s simulated system to the experimental results of [1]. The same parameters were chosen for the simulation, as given by the experimentors (except for two mentioned later); namely 3 to 210 modes, 7 planes, a waist size of 25px where each pixel had the size of $8\mu m$, wavelengths (reduced): 1510nm, 1550nm, 1620nm, box size 11px (for the triangular output boxes). Two parameters were different due to limitations of the neural network program. Firstly, in DONN the x and y dimensions of the phase masks have to be the same (500x500), while in the paper [1] 274x500 pixels were chosen. Secondly, the plane spacing had to be varied, since the minimum spacing allowed in Filipovich’s diffractive neural network is chosen so that the Rayleigh-Sommerfeld diffraction integral can be calculated to a certain extend of correctness. Thus, a plane distance of 4.3cm was chosen instead of 2.5cm in the paper. The first change should have minimal effect on the result, since the necessary light transformation is in the centre of the phase mask (and the extended borders - 500 instead of

274 pixels - have no effect). The second change, increasing the plane spacing, introduces some inaccuracies, as shown in the next section, but should not be of concern for this estimating comparison.

Crosstalk is defined as the ratio of the power in the desired output box to the sum of the power in the wrong output boxes. And the insertion loss as the “average loss over all possible modal superpositions” [1], presumably meaning the total power of light captured outside the desired boxes.

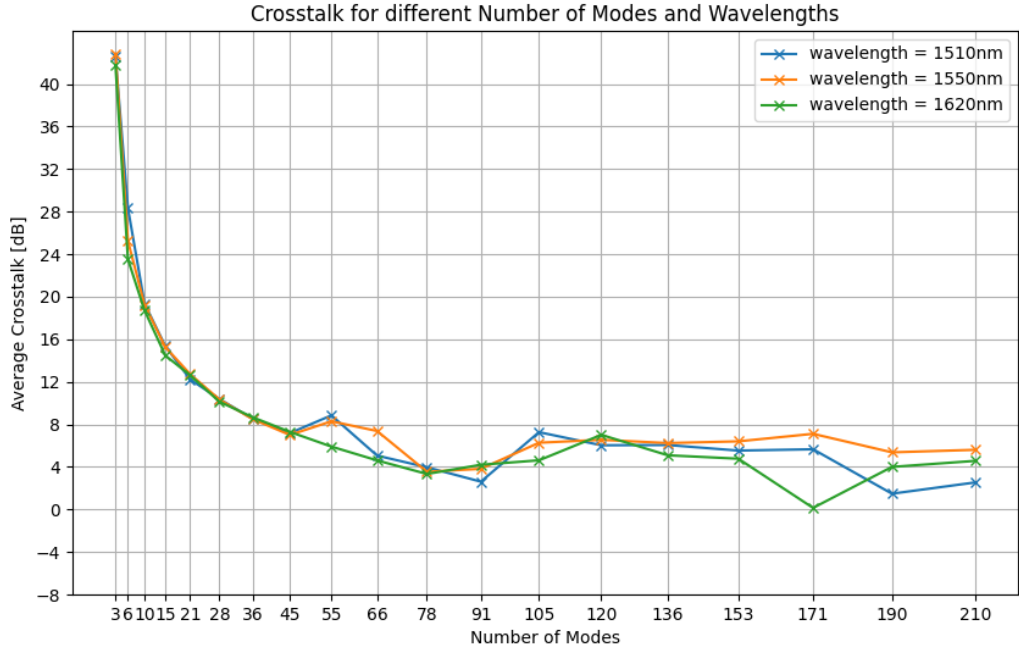


Figure 4.1: Simulation results for crosstalk.

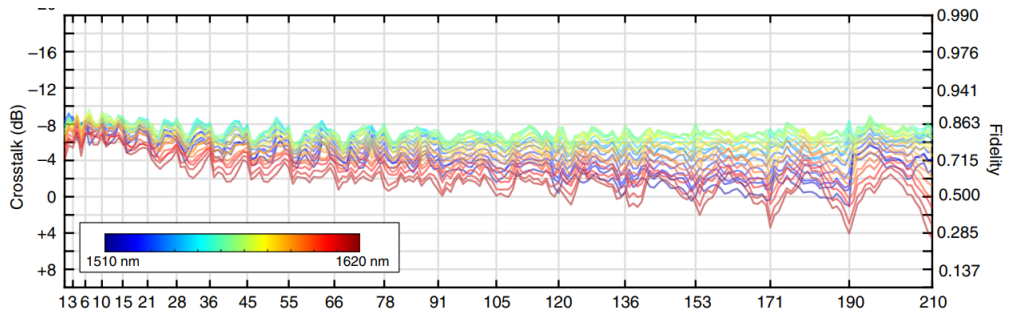


Figure 4.2: Experimental crosstalk results from [1].

Let us interpret the two figures: every crosstalk value in the simulation is positive, while most in the experiment by [1] are negative. For those unfamiliar with the dB scale: the negative dB values represent percentages from 0-100%, while the

positive dB values represent percentages above 100%. Using the definition of crosstalk from above, this shows that in the simulation more energy ended up at the desired output than in the rest of the boxes combined (the opposite is true for the experiment). The superior simulated results need to be verified experimentally, which has not yet been done, but more on this can be read in the next chapter 5.

The high values for crosstalk in the simulation figure at 3, 6 and 10 modes are due to the fact that high mode sorting accuracies (according to the definition of crosstalk above) can be achieved when using the same number of phase masks or more than the number of modes to be sorted. This was not observed in the experiment; the limitations of the experiment for this are unclear.

The comparison of the loss values is also in favour of the simulation, as shown in Fig.4.3 and Fig.4.4. Taken together, these two simulations for crosstalk and loss indicate the validity of the digital diffractive neural network architecture as a whole.

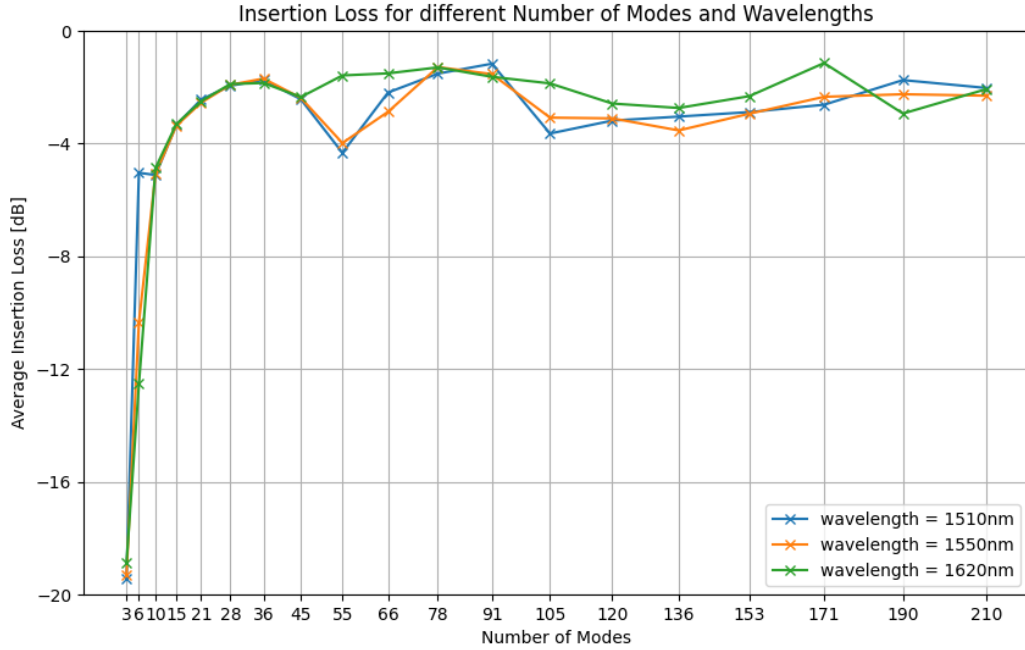


Figure 4.3: Simulation results for loss [dB].

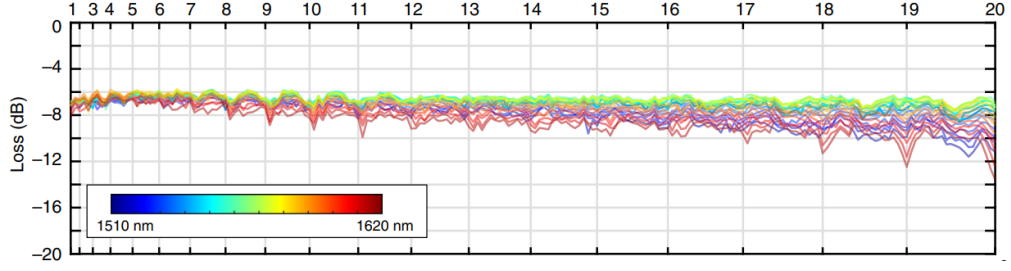


Figure 4.4: Experimental loss results from [1].

4.2 Simulation Results for Different Plane Distances

To further test the physical properties of the system, a simulation was run for different plane distances. The fluctuations in the graph are due to the small number of epochs with which these 2 models (one for each plane distance) were trained - with 1000 or more epochs a smooth curve would be obtained, since the loss function would no longer stumble over local maxima causing the fluctuations. However, the number of epochs was kept small to trade speed against accuracy and verify the assumption of crosstalk dependence on plane distance. The expectation

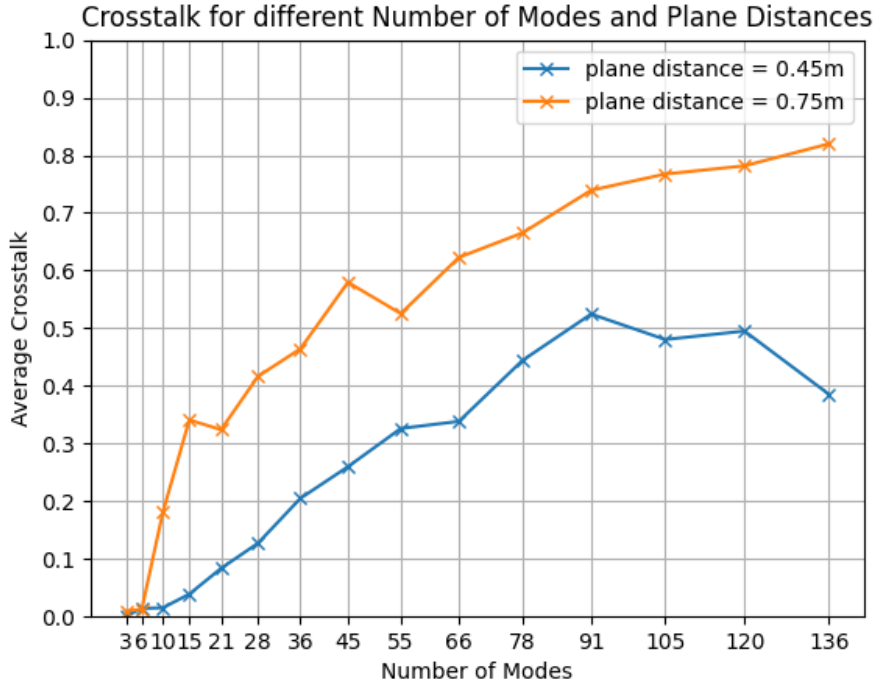


Figure 4.5: Crosstalk for different plane distances.

Simulation Parameters: 5 planes of 700x700 pixels (pixel size: $12.5\mu m$), beam waist 20px, training epochs 300, output box size 10px.

that greater distances would result in higher crosstalk and higher insertion loss

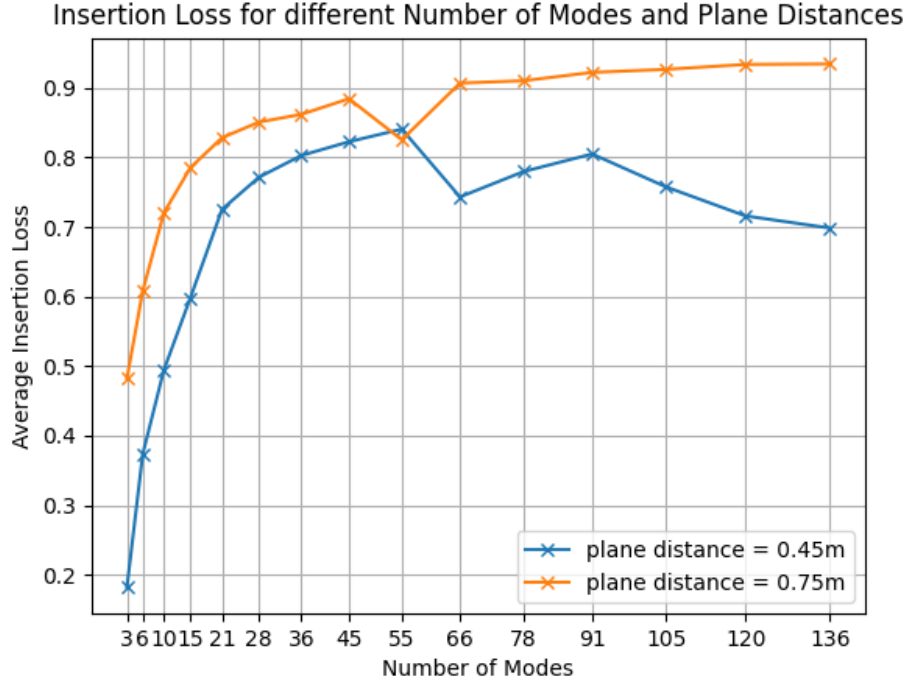


Figure 4.6: Insertion loss for different plane distances.

was met, as the longer distances cause more diffraction of the light field. On the contrary, for close planes, the light cannot propagate as far outwards and tends to terminate close to the right boxes. A similar result can be found in [2].

This result should be kept in mind for further simulations, where the plane spacing had to be increased in some cases (from the original plane spacing of a reference paper) to ensure that the simulation was within the bounds of the correct light propagation distance to compute the Rayleigh-Sommerfeld diffraction integral, as considered in the previous section. This may lead to slightly worse results in my simulations, but should not be decisive.

4.3 Impact of Beam Waist Size on Results

In this section a comparison of simulation results for several beam waist sizes is presented. The purpose of investigating this topic is to find out if there is a sweet spot for the size of the HG mode. One can assume that there is a lower limit for an optimal beam waist, since for very small beam diameters the pixel size becomes a limiting factor for resolution and affects the modifiability of the beam (small beams can be modified by fewer pixels on the phase mask). On the other hand, if the beam is too large, the light is scattered and there are high insertion losses.

The following figures visualise the results of the simulation.

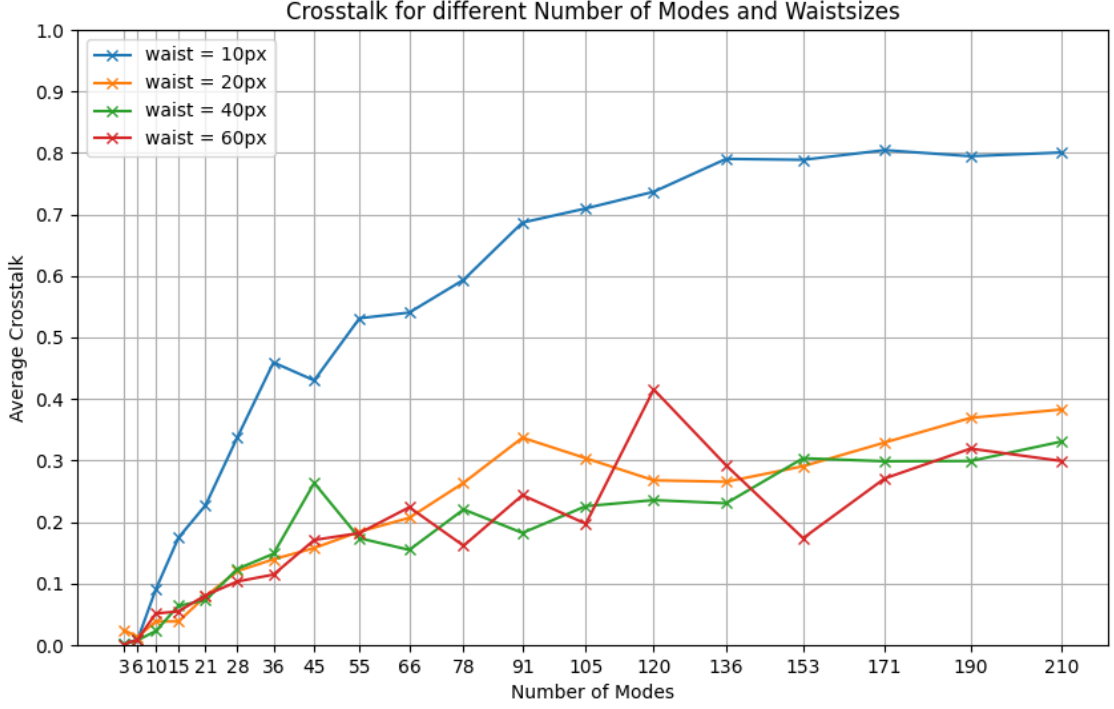


Figure 4.7: Crosstalk values for different beam waists.

Simulation Parameters: 5 planes of 700x700 pixels (pixel size: $12.5\mu m$), wavelength of 1550nm, training epochs 500, fixed output boxes in triangular arrangement.

The effect of the lower limit on beam size is easy to see, but the existence of an upper limit is questionable as the performance is roughly the same for beam waists of 20px, 40px and 60px. It would not make sense to increase the beam size much further, as the propagating beam would become larger than the size of the phase mask, which is not desirable. This suggests that there is no sweet spot for the beam size of the HG mode, but rather a lower limit that should not be crossed.

4.4 Comparison with [2]

A further simulation was carried out for several HG modes and different phase planes and fixed output boxes to compare the performance of the system with [2] by also sorting the modes into “hard boxes” or fixed outputs (soft output boxes will come later). The same parameters were used as in their simulation (except for one, mentioned later), namely a wavelength of 1550nm, a phase mask size of 700 x 700 pixels, and a pixel size of $12.5\mu\text{m}$. In addition, a beam waist of 20px was chosen, as well as an output box size of 10px and a box distance of 5px (these parameters were not specified in [2]). However, one parameter, the plane distance, had to be chosen differently (34.6cm instead of 5cm) due to the limitations of the neural network (Rayleigh-Sommerfeld condition) described in section 4.1.

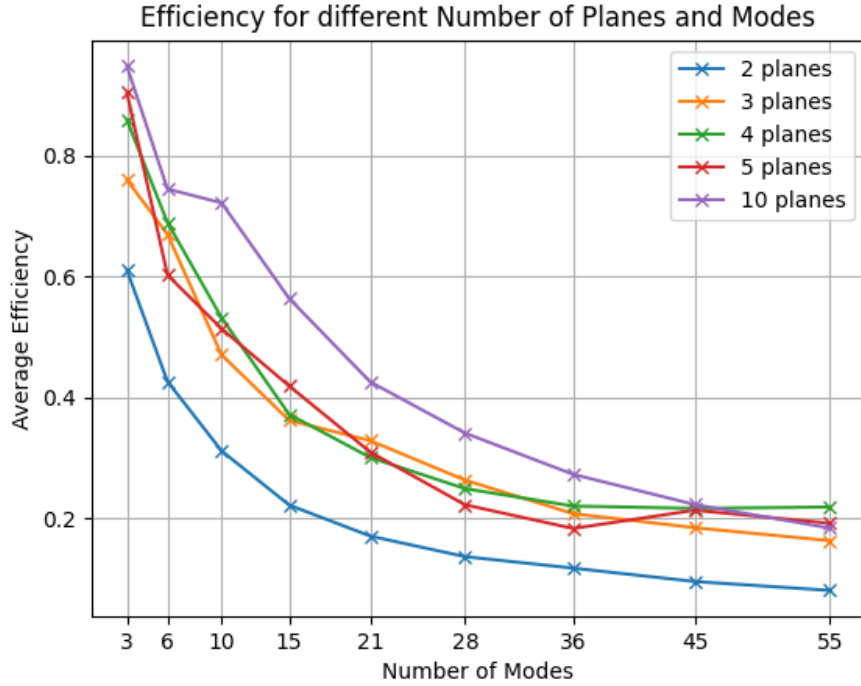


Figure 4.8: Efficiency Values from Simulation

The first two figures (Fig. 4.8 and Fig. 4.9) are a comparison of the efficiency values from my simulation with [2]. Efficiency is given by the ratio of power in an input mode to the power in the desired output box. As can be seen, the efficiency values in the first figure exceed the values in the second figure by a few percentage points, which shows that the diffractive neural network outperforms the WFM for the task of maximising efficiency (note that the DONN was only trained on 400 epochs and already achieves better performance in terms of efficiency). However, the next figures show the underperformance for crosstalk values which will be discussed below.

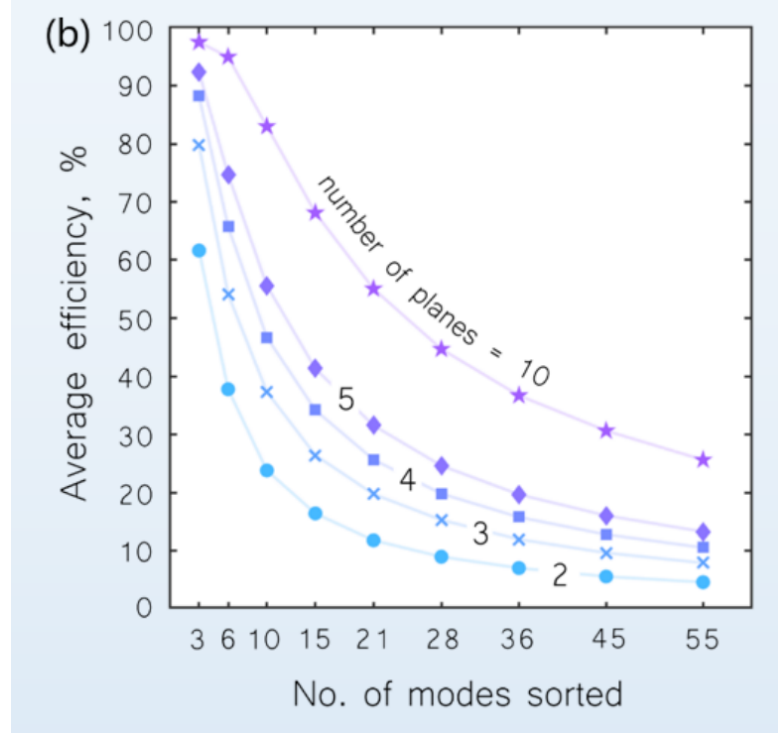


Figure 4.9: Efficiency Values from [2].

In Fig. 4.10 and Fig. 4.11 two simulation results - the results from [2] and my own - are compared in terms of their crosstalk values. Crosstalk is defined differently from 4.1; namely, according to [2], it is the ratio of the sum of the power of all the wrong output boxes to the sum of the power in all the output boxes. Although the results for 2 planes are similar, the crosstalk values for my simulation are worse as the number of planes increases. There are several explanations for this. On the one hand, the waist size may not be optimal, as it can be expected that for small waist sizes the pixel size limits the resolution and thus reduces the efficiency. Nevertheless, no simulation is done with an optimal waist size, as it seems more relevant to move on to free output sorting simulations in the given time, see Sec. 4.5. On the other hand, the increased plane spacing (which had to be chosen under the conditions set by the D²NN) leads to a higher loss, as shown in section 4.2. Another reason could be the small number of training epochs (400), extending the training should give better results. Furthermore, manually selecting each output box for a specific mode would additionally improve accuracy, as observed in small tests. Unfortunately, [2] does not specify how the waist size and output box arrangement were chosen, so no perfect comparison can be made.

Looking at [2]'s simulations, the following question arises: how was the diffraction integral calculated for the small plane distance of 50mm? In the case of Matthew Filipovich's digital diffractive neural network, these lengths are not feasible because

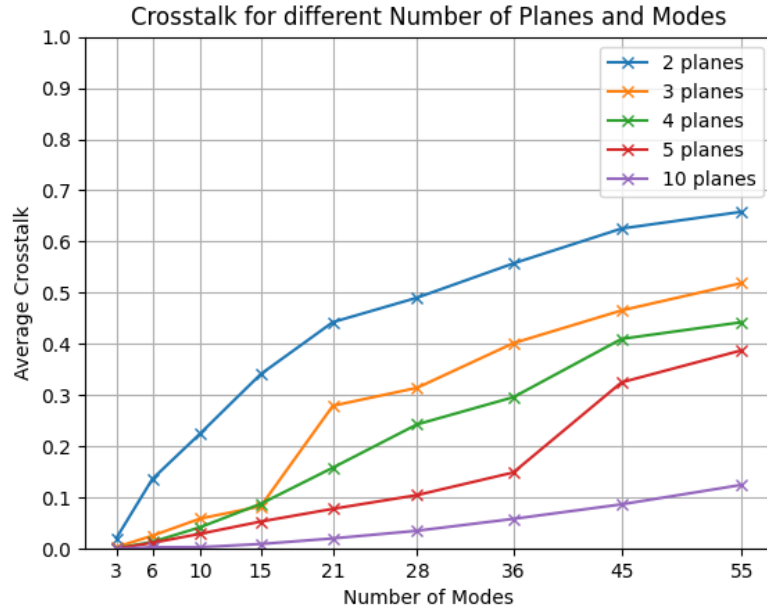


Figure 4.10: Crosstalk Values from Simulation

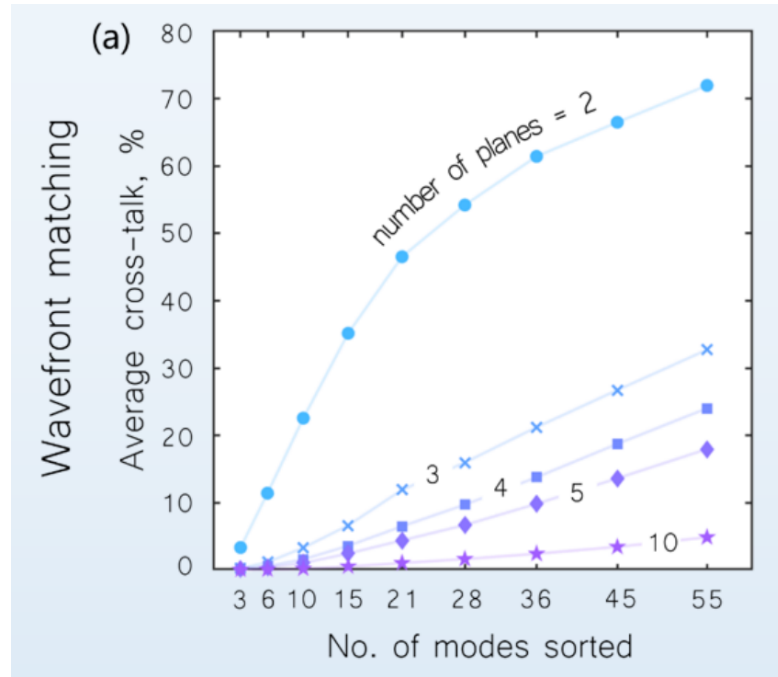


Figure 4.11: Crosstalk Values from [2].

the optical field would not be calculated correctly. Therefore, the results from [2] may not be accurate and a good reference. This should be investigated further in the future to include smaller plane distances in the simulation.

4.5 Free Output Sorting

Earlier the diffractive neural network was used for fixed triangular output masks, see Fig. 4.12. There the system optimized sending light into predefined boxes. In this section the D²NN was designed for sorting into soft boxes, which can be defined by the network itself. The objective of this network is to minimize the overlap of all modes at the output, so that they can be individually detected by e.g. a camera. Thus, the system finds new output arrangements for every model it trains, one such output mask is shown in Fig. 4.13. The results are shown in

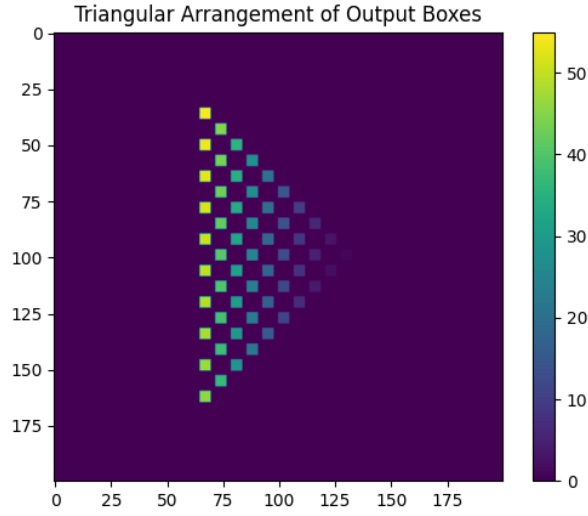


Figure 4.12: Output Mask from Previous Simulations & Reference Papers. Proportions are not to scale and differ in every simulation.

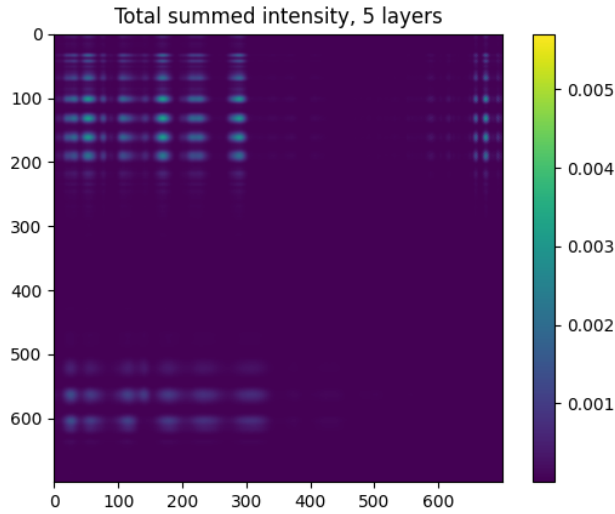


Figure 4.13: Exemplary Output Mask of Free-Sorting D²NN with 5 Layers.

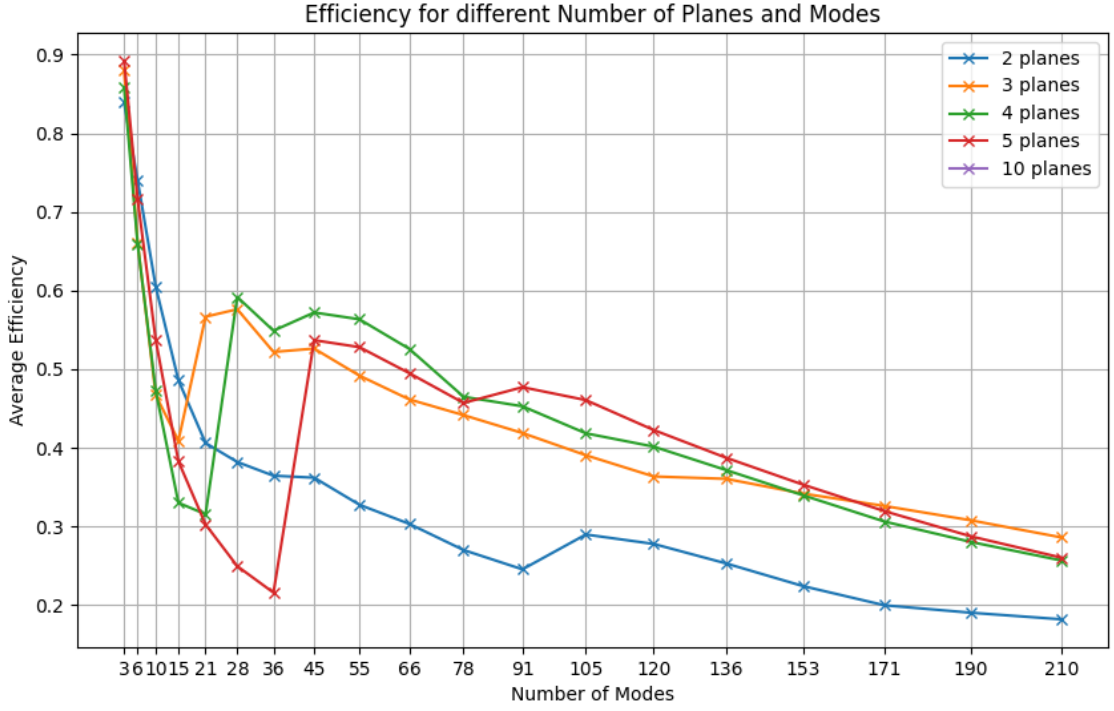


Figure 4.14: Efficiency Values for Soft-Box Simulation

the next figure. When comparing to the results from my previous simulation for

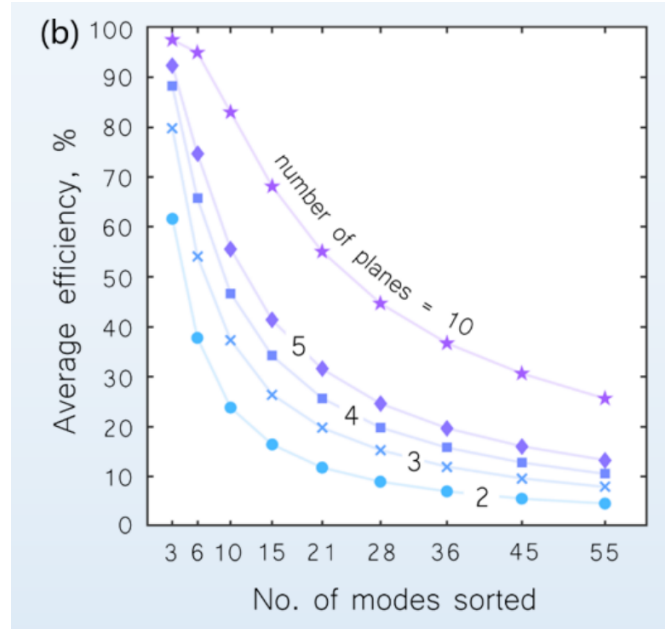


Figure 4.15: Efficiency Values from [2].

fixed output boxes and to [2] in the section above (cf. Fig. 4.9 and Fig. 4.11), one can see that efficiency values are improved further by a factor of 2 or 3, while cross talk also increases, which is not desirable. It seems as though the D^2NN favours

efficiency over crosstalk. While in some experiments this might be advantageous in others it is a downfall. Controlling the trade off between efficiency and crosstalk would be very beneficial. An addition to the loss function would allow just that, which will be explained in the next section.

The characteristic kinks of the orange, red and green curve could be observed in all simulations and cannot be attributed to the flaws of the neural network, such as training for too little epochs or the loss function getting stuck in local minima. Instead this seems to be an inherent property of the system where no explanation could be found.

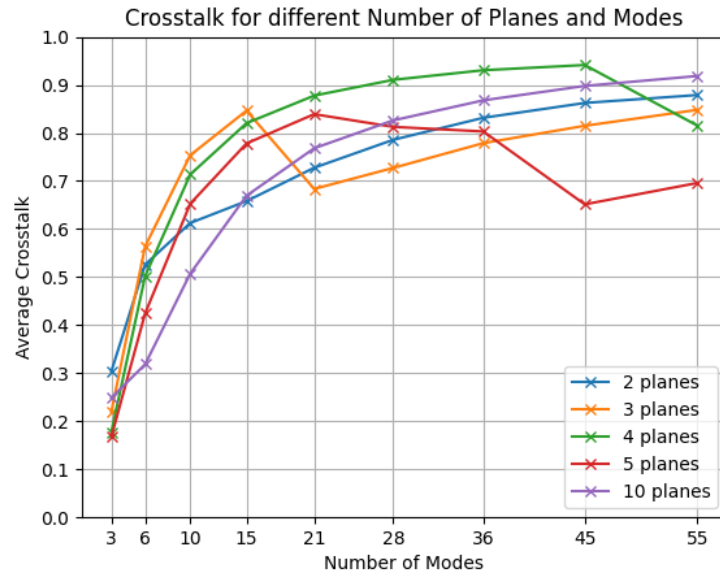


Figure 4.16: Crosstalk Values from soft-box Simulation

4.6 Simulation Outlook

This section discusses several conceptual improvements that can be made to the diffractive neural network to enhance its simulative performance and in the long run also boost the accuracy of the experiment.

First and foremost, the loss function should be extended to include crosstalk and power loss directly in the optimisation problem, rather than just maximising the overlap between the MPLC output modes and the target output modes. The latter would be written as $L = 1 - |\Psi_{MPLC} \cdot \Psi_{Target}|^2$, while the extended loss function can be written as $L = 1 - a|\Psi_{MPLC} \cdot \Psi_{Target}|^2 + b|\Psi_{MPLC} \cdot \Psi_{wrongTargets}|^2 + c|\Psi_{MPLC} \cdot \Psi_{Background}|^2$. The second term describes the crosstalk, as it is the overlap of the MPLC output mode with all the false target modes. The third term describes the energy loss in the system. This direct inclusion of the crosstalk and loss terms in the objective function has resulted in an order of magnitude reduction in inaccuracies in [2].

As seen in the results from the previous section, trading off the balance between efficiency and crosstalk by adjusting the weight parameters a, b, c would lead to a much better performance of the simulated D²NN and seems crucial to compete with similar MPLC mode sorting models in terms of crosstalk.

Secondly, the cost function can be extended by a regularisation term, such as the total variational norm [32]. This would allow the phase masks to be smoothed and thus reduce the crosstalk inherent to the SLM (as opposed to the crosstalk induced by tilted incoming beams). Neighbouring pixels on the SLM should not have very different values, as the voltage applied to one pixel can leak to the others.

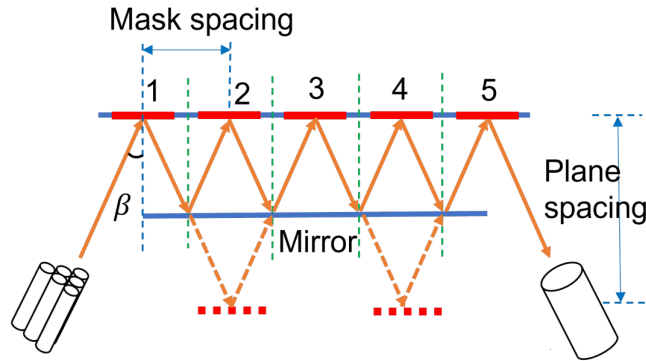


Figure 4.17: MPLC of light between mirror and SLM. The phase plates are encoded into different areas (1,2,3,4,5) of the SLM. Incident Angle β and spacings are shown to compute β . Modified from [16].

Thirdly, a method of stretching the phase mask in the horizontal direction could

be used that takes into account the distortion from the tilted incoming beam [16]. This is particularly important for the setup with beam bounces between an SLM and a mirror, while less important for physical diffraction planes placed one behind the other, as in [11].

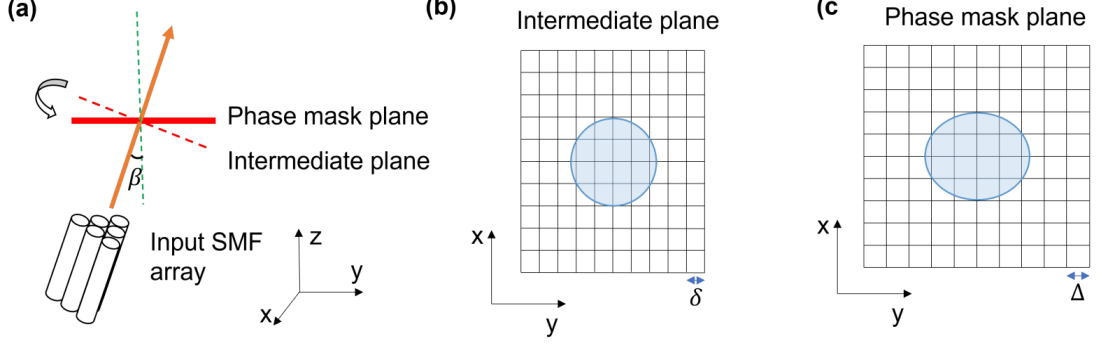


Figure 4.18: Stretched phase mask.

- a) light hits the phase mask at a slight angle β . b) the intermediate plane would be appropriate if the incident angle was 0 degrees. c) when considering the incident angle the stretched phase mask is necessary. Modified from [16].

In the Fig. 4.17 the distances for the calculation of β are shown, $\beta = \arctan \frac{\text{mask spacing}}{\text{plane spacing}}$. The cosine of β can then be used as a scaling factor for the phase mask stretching, see Fig. 4.18, to stretch the circular phase plane into an elliptical shape in the horizontal direction.

Furthermore, a broadband design would improve the performance of multi-plane light conversion tasks [16], since wavelength fluctuations due to cavity imperfections and other defects in the laser are another source of error. This could be implemented by adding another loop to the code for different wavelengths, averaging over different wavelengths around the central wavelengths. This would hopefully make the experiment more robust (cf. [1]) and smooth the phase masks in the wavelength domain.

In addition, training the D²NN on mini-batches of modes seems to be very effective for diffractive neural network architectures (but not fully connected neural networks), as it was heuristically used and shown in [29]. The success was attributed to the lack of non-linear activation functions in the D²NN (which normal artificial neural networks have) after running empirical tests.

Finally, a parameter for the size of the input mode could be added to the optimisable parameters of the neural network by including HG mode generation (with a certain beam waist parameter) in the function of the diffractive network. In this way, the correct ratio of phase masks to HG modes could be found, which should verify earlier results in section 4.3. Note that the phase mask size is fixed by the

limited size of the SLM, but the mode size is a free parameter.

Once the correct mode size has been found, the neural network can be trained with slightly different mode sizes. This can be considered as part of training on noisy data, but is certainly essential to improve the stability and accuracy of the system in the experiment.

CHAPTER 5

EXPERIMENT

5.1 Setup

A simplified version of the optical setup used in this project is shown in Fig. 5.1. The incoming laser beam, seen in the lower right corner, is expanded by two convex lenses placed at a distance of two focal lengths ($f_1 + f_2$, $f_2 > f_1$) from each other. The beam is then reflected by a Polarizing Beam Splitter (PBS) which

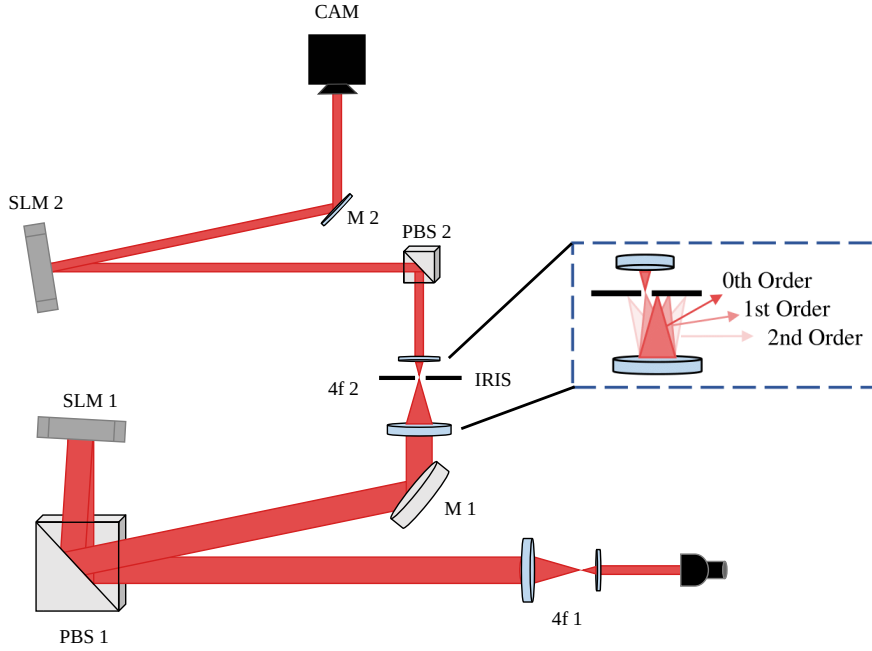


Figure 5.1: Simplified Experimental Setup.

The light field enters from the bottom right, two lenses magnify the beam and it hits the spatial light modulator, where an HG mode is projected onto its wavefront. The beam then shrinks in diameter and reflects off a second SLM, which sorts the mode, after which it is detected by the camera.

filters out the vertical polarisation of the beam (necessary for the SLM to work

(see subsection 5.2.2)). When the light hits the first SLM, a pattern is encoded in the optical field in the phase domain. SLM screens are typically made from liquid crystals, and by controlling the electrical current to each pixel, the refractive index of the crystal volume above the pixel is changed, thus modifying the phase of the light on the pixel. In this way, the entire wavefront of the incoming light wave can be shaped, pixel by pixel.

After the first SLM, the wavefront propagates to the next 4f system, where the beam diameter decreases again and the iris filters out the first order of the beam. The first order is needed because the SLM can create amplitude modulations of the light field in this order (and not in the 0th order [33]). The different orders, however, are created by a diffraction grating on the SLM, which splits the beam, as shown in Fig. 5.1 on the right in the blue dashed box.

At the second PBS, the light is redirected to the next SLM, where the wavefront is modified again. Finally, the beam hits the camera and is captured. In this simplified experimental setup, only one plane light conversion takes place at the second SLM - the first spatial light modulator is used only to create the desired input field: in this case a HG mode. The mode is then sorted by a specific phase modulation of the plane, which redirects the light of various modes to different areas of the detection plane.

A more complex version of the experimental setup is shown in figure 5.2. No details have been omitted as in the previous figure. New features are the collimating lens, the half-wave plates and the mirrors - the details for each component are given in the description of the figure.

Note that the seventh mirror (M7) was not yet in place during the preliminary experiments carried out during my stay with the group. The current setup (without M7) should allow sorting a small number of modes with a single reflection on the second SLM acting as a single phase mask. In the long run, however, more diffraction planes are needed to perform a more sophisticated light transformation and sort more modes. Therefore, the seventh mirror will soon be placed to reflect the light field between the mirror and the second SLM several times and to programme individual phase planes on different regions of the SLM to act as diffractive layers, see Fig. 5.2.

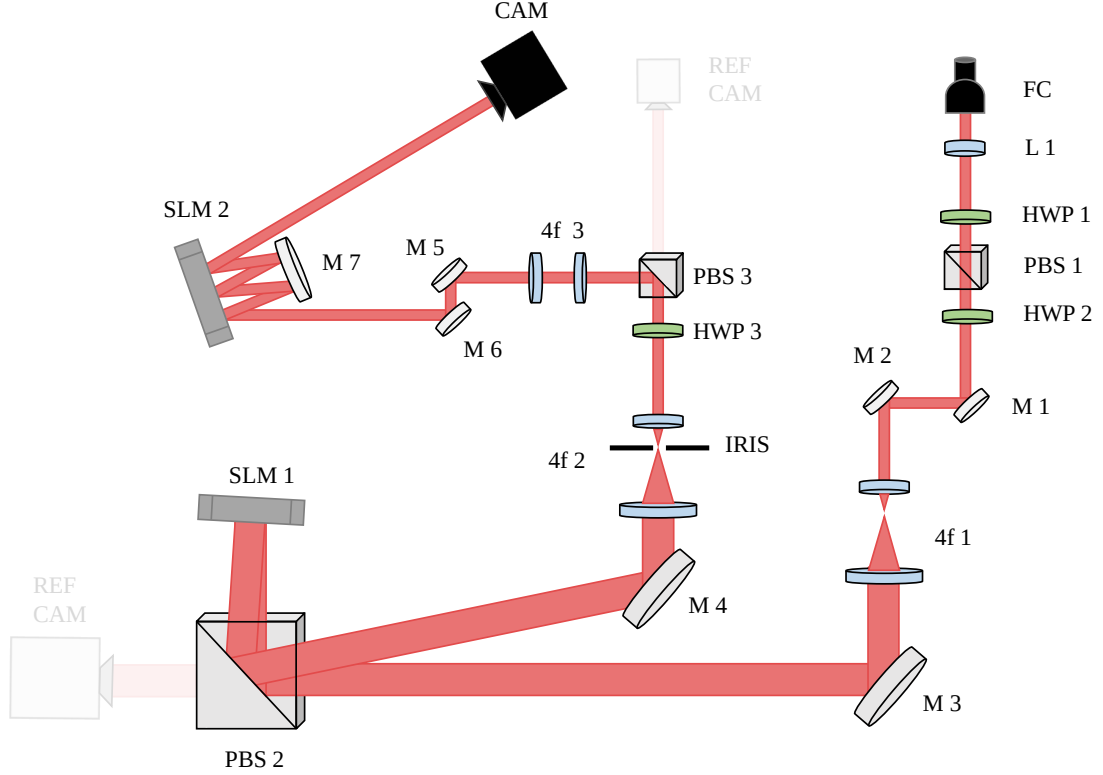


Figure 5.2: Experimental Setup.

This setup includes various additions: The first lense allows better collimation of the beam after the fiber coupler. Turning of the Half Wave Plate (HWP) together with the PBS allows tuning of the intensity. Two mirrors enable the two necessary degrees of freedom for adjusting the beam's direction in space. Further, the reference cameras are used to check the quality of the laser beam and the input modes created on SLM 1 and the third 4f system is useful for fine-tuning the beam collimation. Various bounces off the second SLM perform a multi-plane light conversion for sorting HG modes into different detection areas.

5.2 Techniques

Various experimental techniques are summarized in this section to give a better understanding of the optical setup and identify both potential advantages and error sources.

5.2.1 Polarization Calibration

Fluctuations in intensity detected by the camera suggest that the coupling into the fibre that carried the light to this experiment (not shown in the figure above) was not well adjusted. The laser itself was not the cause of the high fluctuations, as direct measurements of the source were made. The fluctuations were caused by temperature differences, which could be easily verified by holding the fibre in one's hand.

The fundamental reason for these fluctuations is that the polarisation of the incoming beam is not well aligned with one of the two axes of the refractive index of the fibre core material, so the polarisation of the outgoing beam oscillates randomly and this causes intensity fluctuations at the camera because the PBSs only transmit one specific polarisation. The relative refractive index changes caused by the temperature differences must be minimised by aligning the polarisation of the incoming beam parallel to one of the refractive index axes. In this case, no light is polarised parallel to the other refractive index axis, and if that refractive index is relatively changed by temperature, there is no effect on the overall polarisation of the light. This can be thought of analogously to half wave plates. The polarisation of any beam can be decomposed into two orthogonal base polarisations of the material of the HWP. If the refractive index changes for only one of the two polarisation axes, the phase of the beam is shifted and it's polarisation rotates. To avoid this rotation, the polarisation of the incoming beam is aligned along only one axis, so that it is free of intensity fluctuations.

The polarisation calibration involved using a teacup (hence 'tea test') to apply a temperature gradient to the fibre and cause the intensity to fluctuate due to polarisation changes. A half-wave plate in front of the fibre coupler was rotated to align the polarisation axis of the beam with one of the refractive index axes of the fibre by observing the reduction in signal oscillations.

5.2.2 SLM Calibration

The spatial light modulators are a central part of the setup and therefore need to be well characterised and correctly placed. The following section explains some of the calibration procedures that were carried out.

Firstly, it was necessary to ensure that the light beam was polarised in a way that is optimal for use with the SLMs. In our case, both SLMs require vertically polarised light. This is because the liquid crystals inside the SLM rotate in the vertical plane, affecting the V-polarised oscillations of the electromagnetic field. There is hence a phase delay when the voltage applied to each pixel is changed, causing the liquid crystals to rotate and the refractive index of the medium to alter, which in turn shifts the optical path length and thus the phase. A great intuitive visualisation of this phenomenon can be found here [34]. The correct incoming polarisation is easily achieved by ensuring that the polarising beamsplitter is in the correct orientation.

The SLM then needs to be fully illuminated to produce an optimal image. The

correct beam size was therefore chosen using the first 4f system and the beam was placed on the rectangular SLM screen so that the corners almost touched the beam perimeter.

The beam's angle of incidence was kept to a minimum (less than 5 degrees) so as to induce as little pixel crosstalk as possible when the beam passes through different pixels.

Each SLM comes with its own look-up table (LUT) and calibration file from the manufacturer. Because some pixels on the SLM screen are not as responsive as others, the LUT is needed to change the voltage applied to each pixel by a certain factor to produce the desired result. The calibration file, on the other hand, is designed to compensate for other design flaws, namely not having a smooth, constant phase screen on the SLM, but rather something like what is shown in Fig. 5.3a, where the darkest areas have a phase of $-\pi$ and the yellow areas have a phase of $+\pi$. This calibration file adds small phase shifts to each pixel to create the actual desired image in phase space. The manufacturer's calibration file is shown on the left in the following figure. Next to it is another calibration file, previously obtained by another colleague using an interferometric measurement (Twyman-Green interferometer), which allows the phase distortions of the wavefront of the same SLM to be reconstructed.

Attempting to calibrate the SLM ourselves was not considered a priority, as a colleague had advised us to use the manufacturer's correction file because it is accurate enough. Therefore, the manufacturer's correction file was used in the first instance, and the second correction file from the right figure was kept nearby for future testing and comparing of the two calibration files. Nevertheless, a proper calibration might be performed in the future if this appears to be a source of error.



(a) Manufacturer's Correction.
Supplied by the Producer.



(b) SLM Phase Correction File.
Taken from [35].

5.2.3 Generation of HG modes

The Python code used to generate the HG modes is based on a module called HermiteGaussianE by Andri M. Gretarsson in 2003. The summarising macro code is attached in the appendix. In this way, images of Hermite Gaussian modes in the phase domain were generated and displayed on the SLM, producing a laser beam with the desired amplitude and phase of a given HG mode, as described in the following paragraph.

The grating applied to the LC-SLM in step 2 mentioned in the code is essential to generate the beam with the correct amplitude and phase, $\Psi(x) = A(x)e^{i\Phi(x)}$. The SLM we used can only perform phase shifts $E(x) = e^{i\Phi(x)}$ on individual pixels, but there is a way to implement phase gratings on the SLM so that amplitude modulations can be generated. One way to generate a desired amplitude and phase is to modulate the phase grating height and offset via $\Phi_{gr}(x) = (1 + \text{sinc}^{-1}(A(x)))(\Phi(x) + u_0x)$, cf. Method C in [36] (this paper also includes other ways to achieve amplitude modulation with a phase-only SLM) and [37]. This creates the desired mode in the first order of diffraction, which must be filtered to obtain the desired optical field.

The functions used to create these modes and gratings include `amp_and_phase()`, `make_img()`, which were collected from modules of other experiments by James Spall.

5.2.4 Python Interface

A Python interface has been created to automate the measurement process. Essentially, it controls which images are placed on each SLM, which input modes are created and which sorting phase plates are used. In addition, a uEye Python module can be used to automatically acquire images (`camera_on()`, `save_frame()`). The SLM control module was written by Sebastien Popoff and contains the functions `SLMdisplay()` and `updateArray()` [38]. The macro code can also be found in the appendix.

CHAPTER 6

CONCLUSION

This report firstly covered artificial neural networks and my own simulation results necessary for the understanding and training of diffractive neural networks in later chapters. Secondly, it presented an analysis of the status quo of mode sorting by multi-plane-light-conversion, firstly explaining artificial neural networks and MPLC itself and discussing the common algorithm for phase plate generation, the wavefront matching method. Then, an alternative method using deep diffractive neural networks was described and its many advantages illustrated. Afterwards, I showed the performance of the simulated diffractive neural network and visualised a lower crosstalk and insertion loss than [1]. Furthermore, the increased efficiency for mode sorting compared to the wavefront matching method of [2] was presented, as well as the poor crosstalk performance. All these results were discussed in detail, and a whole section was written about many ideas for improving the system. Finally, the experiment was demonstrated along with the steps taken so far. This project will then be developed further in the coming months by the Quantech group at University of Oxford.

ACKNOWLEDGEMENT

I would like to express my deep regard to Alex Lvovsky for giving me the opportunity to work in his group, providing guidance throughout the works of this project, as well as allowing me to come to Oxford to experience the rich intellectual culture and enabling this unforgettable stay.

I also take this opportunity to express my profound gratitude to all Quantech Group members, Matthew F., Alexander D., Aleksei M., James S., Kaden B., Jernej F. and Mert E. for the many discussions on theory and experiment, as well as the collegiality and always good time.

APPENDIX

A.1 Macro Code for Generation of HG modes (Sec. 5.2.3)

```
# 0: set your beam parameters
image_height = ...
image_width = ...
wavelength = ...
pixel_size = ...
q = ... #complex beam parameter
...

# 1: create HG modes with Hermite polynomials via the function
# HermiteGaussianE_Field

for i in m:
    for j in n:
        HG_modes[i,j,:,:] = HermiteGaussianE_Field(x, y, m[i],n[j], q);

# 2: put grating on HG modes via make_img() function from James' module

for i in range(np.shape(E)[0]):
    for j in range(np.shape(E)[1]):
        mode_wanted = E[i,j,:,:]
        amplitude,phase = amp_and_phase(mode_wanted)
        #normalizes amplitude and phase
        SLM_mode_mn[i,j,:,:] = make_img(amplitude, phase)

# 3: convert HG modes to .bmp files for SLM
...
```

A.2 Macro Code for Python Interface (Sec. 5.2.4)

```

# 0: set SLM parameters
slm_w = int(1920)
slm_h = int(1152)

# 0.1: initialize the SLM display
slm = SLMdisplay([corner_coor], [slm_w], [slm_h], ['SLM1'], isImageLock=False)

# 0.2: load the HG modes and add the correction file to the HG modes
for m in m_indices:
    for n in n_indices:
        HG_modes[m,n,:,:] = plt.imread(directory + filename)

slm_corr = plt.imread(correction_file)

for m in m_indices:
    for n in n_indices:
        HG_modes_corr[m,n, :, :] = HG_modes[m,n, :, :] + slm_corr

# 1: display HG mode on first SLM

slm.updateArray('SLM1', HG_modes_corr[0,1])

# 2: generate x phase plates on second SLM (here x = 3)

phase_plates = np.array([mask1, mask2, mask3])
slm_corr_2 = plt.imread(correction_file_2) # load correction file 2

slm_plates = np.zeros((1152, 1920)) # initialize slm display

slm_plates[326:826, 210 - gap : 710 - gap] = phase_plates[0]
slm_plates[326:826, 710 : 1210] = phase_plates[1]
slm_plates[326:826, 1210 + gap : 1710 + gap] = phase_plates[2]

slm_plates += slm_corr_2.astype('float64') # add correction file to
                                           #phase plates

slm.updateArray('SLM2', slm_plates) # display on SLM 2

# 3: take camera images

camera_on(EXPOSURE_TIME_MS, FRAME_RATE)

save_frame('file_path', png_name)

```

REFERENCES

- [1] N. K. Fontaine, R. Ryf, H. Chen, D. T. Neilson, K. Kim, and J. Carpenter, “Laguerre-gaussian mode sorter,” *Nature Communications*, vol. 10, Apr 2019.
- [2] H. Kupianskyi, S. A. R. Horsley, and D. B. Phillips, “High-dimensional spatial mode sorting and optical circuit design using multi-plane light conversion,” 2022.
- [3] A. A. Pushkina, G. Maltese, J. I. Costa-Filho, P. Patel, and A. I. Lvovsky, “Superresolution linear optical imaging in the far field,” *Phys. Rev. Lett.*, vol. 127, p. 253602, Dec 2021.
- [4] K. K. M. Bearne, Y. Zhou, B. Braverman, J. Yang, S. A. Wadood, A. N. Jordan, A. N. Vamivakas, Z. Shi, and R. W. Boyd, “Confocal super-resolution microscopy based on a spatial mode sorter,” *Opt. Express*, vol. 29, pp. 11784–11792, Apr 2021.
- [5] H. Bechmann-Pasquinucci and W. Tittel, “Quantum cryptography using larger alphabets,” *Phys. Rev. A*, vol. 61, p. 062308, May 2000.
- [6] M. Mirhosseini, O. S. Magaña-Loaiza, M. N. O’Sullivan, B. Rodenburg, M. Malik, M. P. J. Lavery, M. J. Padgett, D. J. Gauthier, and R. W. Boyd, “High-dimensional quantum cryptography with twisted light,” *New Journal of Physics*, vol. 17, p. 033033, mar 2015.
- [7] G. Gibson, J. Courtial, M. J. Padgett, M. Vasnetsov, V. Pas’ko, S. M. Barnett, and S. Franke-Arnold, “Free-space information transfer using light beams carrying orbital angular momentum,” *Opt. Express*, vol. 12, pp. 5448–5456, Nov 2004.
- [8] N. Bozinovic, Y. Yue, Y. Ren, M. Tur, P. Kristensen, H. Huang, A. E. Willner, and S. Ramachandran, “Terabit-scale orbital angular momentum mode division multiplexing in fibers,” *Science*, vol. 340, no. 6140, pp. 1545–1548,

2013.

- [9] J. Spall, X. Guo, T. D. Barrett, and A. I. Lvovsky, “Fully reconfigurable coherent optical vector–matrix multiplication,” *Opt. Lett.*, vol. 45, pp. 5752–5755, Oct 2020.
- [10] J. Spall, X. Guo, and A. I. Lvovsky, “Hybrid training of optical neural networks,” 2022.
- [11] X. Lin, Y. Rivenson, N. T. Yardimci, M. Veli, Y. Luo, M. Jarrahi, and A. Ozcan, “All-optical machine learning using diffractive deep neural networks,” *Science*, vol. 361, pp. 1004–1008, sep 2018.
- [12] M. Nielsen, “Neural networks and deep learning,” 2015.
- [13] [https://en.wikipedia.org/w/index.php?title=MNIST_database&oldid=1156773508; accessed 21-June-2023].
- [14] A. Labach, H. Salehinejad, and S. Valaee, “Survey of dropout methods for deep neural networks,” 2019.
- [15] [https://iq.opengenus.org/content/images/2023/01/2023_01_20_0te_Kleki-min.png; accessed 06-July-2023].
- [16] Y. Zhang and N. K. Fontaine, “Multi-plane light conversion: A practical tutorial,” 2023.
- [17] B. J. Puttnam, G. Rademacher, and R. S. Luís, “Space-division multiplexing for optical fiber communications,” *Optica*, vol. 8, pp. 1186–1203, Sep 2021.
- [18] L. Garcia, O. Pinel, N. B. P. Jian, L. Jaffrs, J. F. Morizur, and G. Labroille, “Fast adaptive laser shaping based on multiple laser incoherent combining,” 2017.
- [19] S. Bade, B. Denolle, G. Trunet, D. Allioux, P. Jian, O. Pinel, and G. Labroille, “Fabrication and characterization of a mode-selective 45-mode spatial multiplexer based on multi-plane light conversion,” 2019.
- [20] R. Fickler, F. Bouchard, E. Giese, V. Grillo, G. Leuchs, and E. Karimi, “Full-field mode sorter using two optimized phase transformations for high-dimensional quantum cryptography,” *Journal of Optics*, vol. 22, p. 024001, jan 2020.

- [21] Y. Zhou, M. Mirhosseini, S. Oliver, J. Zhao, S. M. H. Rafsanjani, M. P. J. Lavery, A. E. Willner, and R. W. Boyd, “Using all transverse degrees of freedom in quantum communications based on a generic mode sorter,” *Optics Express*, vol. 27, p. 10383, mar 2019.
- [22] A. Mair, A. Vaziri, G. Weihs, and A. Zeilinger, “Entanglement of the orbital angular momentum states of photons,” *Nature*, vol. 412, pp. 313–316, jul 2001.
- [23] S. Leedumrongwatthanakun, L. Innocenti, H. Defienne, T. Juffmann, A. Ferraro, M. Paternostro, and S. Gigan, “Programmable linear quantum networks with a multimode fibre,” *Nature Photonics*, vol. 14, pp. 139–142, dec 2019.
- [24] F. Brandt, M. Hiekkamäki, F. Bouchard, M. Huber, and R. Fickler, “High-dimensional quantum gates using full-field spatial modes of photons,” *Optica*, vol. 7, p. 98, jan 2020.
- [25] Y. Luo, Y. Zhao, J. Li, E. Çetintaş, Y. Rivenson, M. Jarrahi, and A. Ozcan, “Computational imaging without a computer: seeing through random diffusers at the speed of light,” *eLight*, vol. 2, jan 2022.
- [26] U. G. Būtaité, H. Kupianskyi, T. Čižmár, and D. B. Phillips, “How to build the optical inverse of a multimode fibre,” 2022.
- [27] Y. Zhang, H. Wen, A. Fardoost, S. Fan, N. K. Fontaine, H. Chen, P. L. Likamwa, and G. Li, “Simultaneous sorting of wavelengths and spatial modes using multi-plane light conversion,” 2020.
- [28] N. Fontaine, J. Carpenter, S. Gross, S. Leon-Saval, Y. Jung, D. Richardson, and R. Amezcua-Correa, “Photonic lanterns, 3-d waveguides, multiplane light conversion, and other components that enable space-division multiplexing,” *Proceedings of the IEEE*, vol. 110, pp. 1821–1834, Nov. 2022. Version archived for private and non-commercial use with the permission of the author/s and according to publisher conditions. For further rights please contact the publisher.
- [29] Z. Zhu, J. H. Doerr, G. Li, and S. Pang, “A physical neural network training approach toward multi-plane light conversion design,” 2023.
- [30] N. K. Fontaine, H. Chen, M. Mazur, L. Dallachiesa, K. Kim, R. Ryf, D. Neilson, and J. Carpenter, “Hermite-gaussian mode multiplexer supporting 1035 modes,” in *Optical Fiber Communication Conference (OFC) 2021*, p. M3D.4,

Optica Publishing Group, 2021.

- [31] G. Labroille, N. Barré, O. Pinel, B. Denolle, K. Lenglé, L. Garcia, L. Jaffrès, P. Jian, and J.-F. Morizur, “Characterization and applications of spatial mode multiplexers based on Multi-Plane Light Conversion,” *Optical Fiber Technology*, vol. 35, pp. 93–99, Feb. 2017.
- [32] A. Beck and M. Teboulle, “Fast gradient-based algorithms for constrained total variation image denoising and deblurring problems,” *IEEE Transactions on Image Processing*, vol. 18, no. 11, pp. 2419–2434, 2009.
- [33] E. Bolduc, N. Bent, E. Santamato, E. Karimi, and R. W. Boyd, “Exact solution to simultaneous intensity and phase encryption with a single phase-only hologram,” *Opt. Lett.*, vol. 38, pp. 3546–3549, Sep 2013.
- [34] [<https://www.youtube.com/watch?v=-KzEcJg1wWM>; accessed 11-July-2023].
- [35] Rohith, “Design, fabrication & analysis of liquid immersed phase masks for wavefront correction, master dissertation project,” 2022.
- [36] T. W. Clark, R. F. Offer, S. Franke-Arnold, A. S. Arnold, and N. Radwell, “Comparison of beam generation techniques using a phase only spatial light modulator.,” *Optics express*, vol. 24 6, pp. 6249–64, 2016.
- [37] J. Spall, “Personal notes on lc-slm phase grating theory for beam generation.,” 2023.
- [38] [<https://github.com/wavefrontshaping/slmPy/tree/master>; accessed 17-July-2023].
- [39] [[https://preview-assets-us-01.kc-usercontent.com/210eb585-dc2c-0008-b663-73b9ea5cff38/413ad8ec-6d45-4ce5-8e24-32bb3dc70935/Manual%20\(DVI%20Controller\).pdf](https://preview-assets-us-01.kc-usercontent.com/210eb585-dc2c-0008-b663-73b9ea5cff38/413ad8ec-6d45-4ce5-8e24-32bb3dc70935/Manual%20(DVI%20Controller).pdf); accessed 10-July-2023].